Lab 4: Microsoft Fabric End-to-End Optimization & Governance Lab

This version includes:

- Exact UI actions
- What to click
- What to enter
- Expected output after every step
- Validation checkpoints
- Common issues
- Enterprise best practices

LAB OVERVIEW

Goal

You will build:

CSV Files

↓

Lakehouse

↓

Pipeline

↓

Warehouse

↓

Governance

↓

Access Controls

↓

Lineage Tracking

Step 1 — Verify Workspace Configuration - Open Existing Workspace

Actions

From left panel:

Workspaces

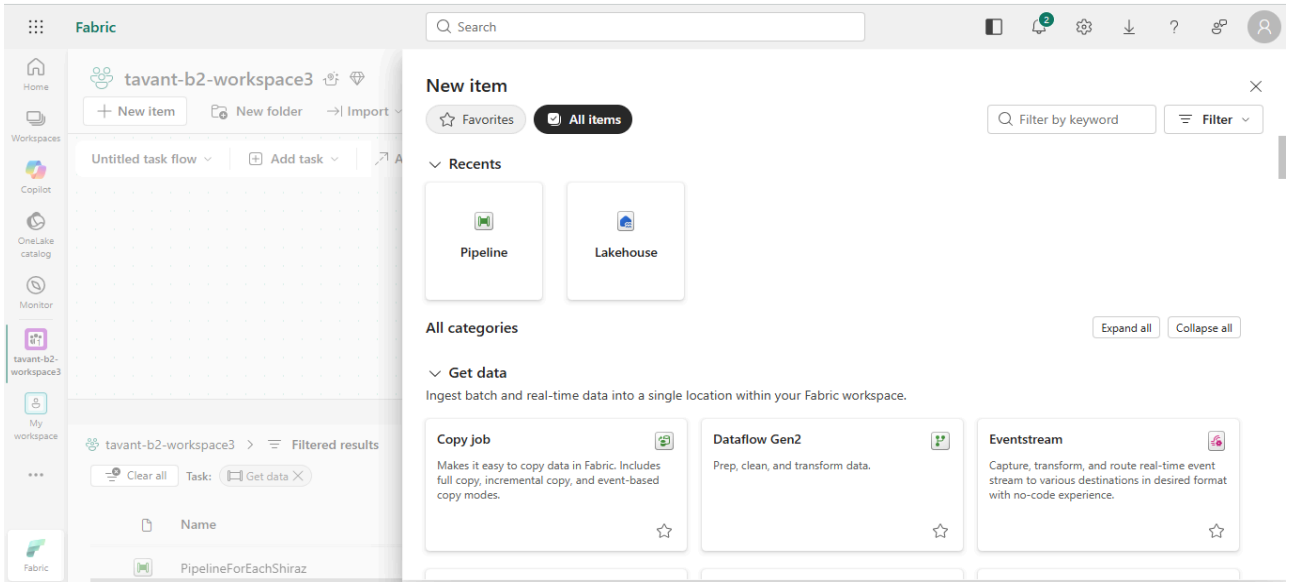
Open your workspace:

tavant-b2-workspace3

Expected Output

You should see:

Item	Expected
New Item button	Visible
Lakehouse option	Available
Data Pipeline	Available
Warehouse	Available



Step 2 — Create Lakehouse

Actions

Click:

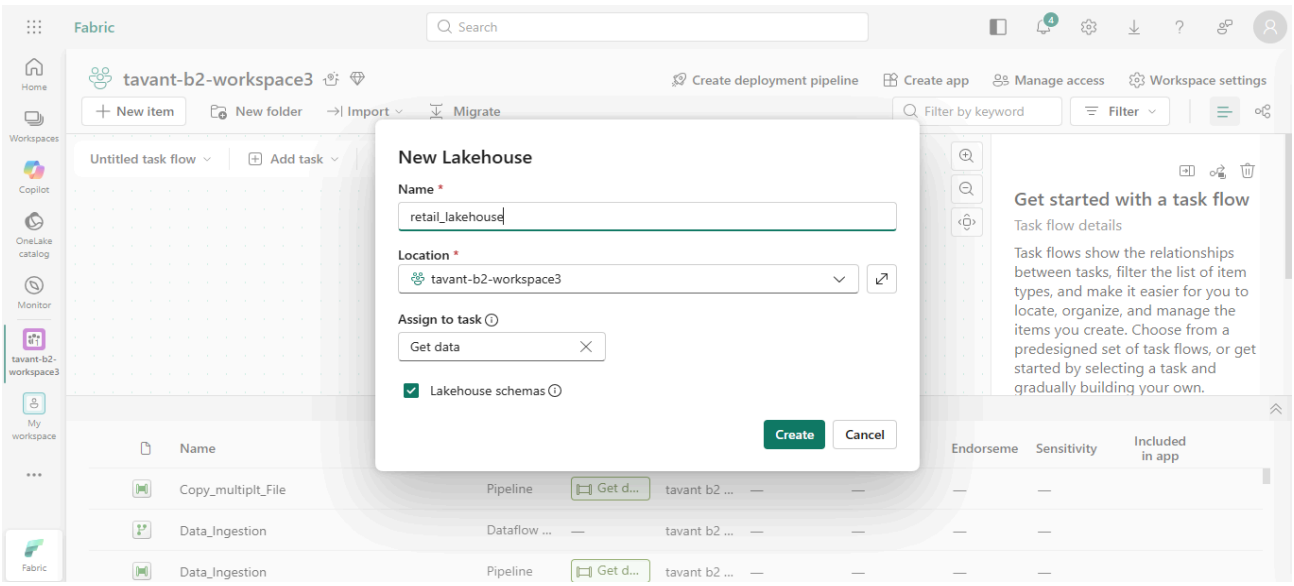
New Item → Lakehouse

Fill:

Field	Value
Name	retail_lakehouse
Lakehouse schemas	Checked

Click:

Create



Expected Output

Lakehouse opens successfully.

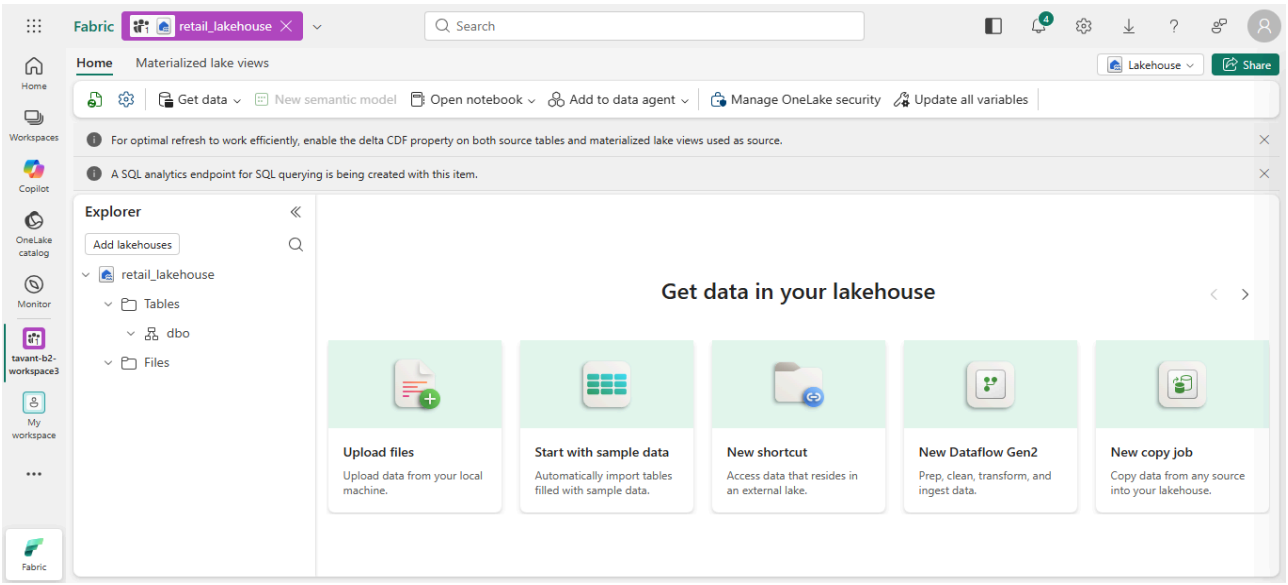
You should see:

Files

Tables

SQL analytics endpoint

Default semantic model



Step 3 — Upload Files

Actions

Inside Lakehouse:

Files → Upload → Upload Files

Upload:

- customers.csv
- products.csv
- sales.csv

Expected Output

Files appear under:

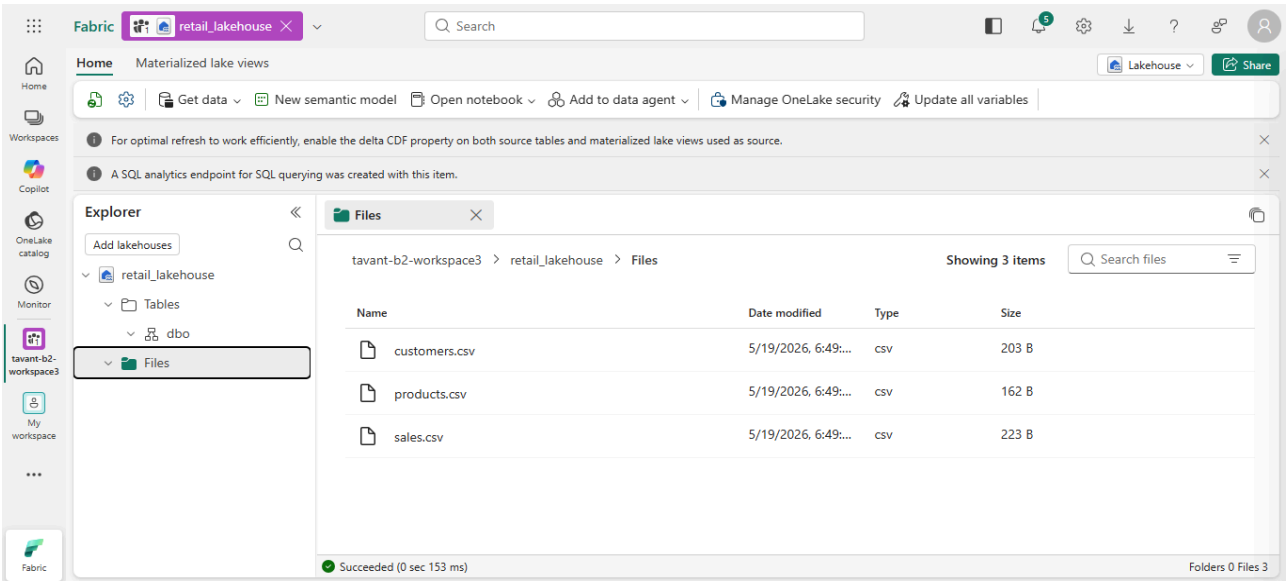
Files

Example:

customers.csv

products.csv

sales.csv



Step 4 — Create Data Pipeline

Actions

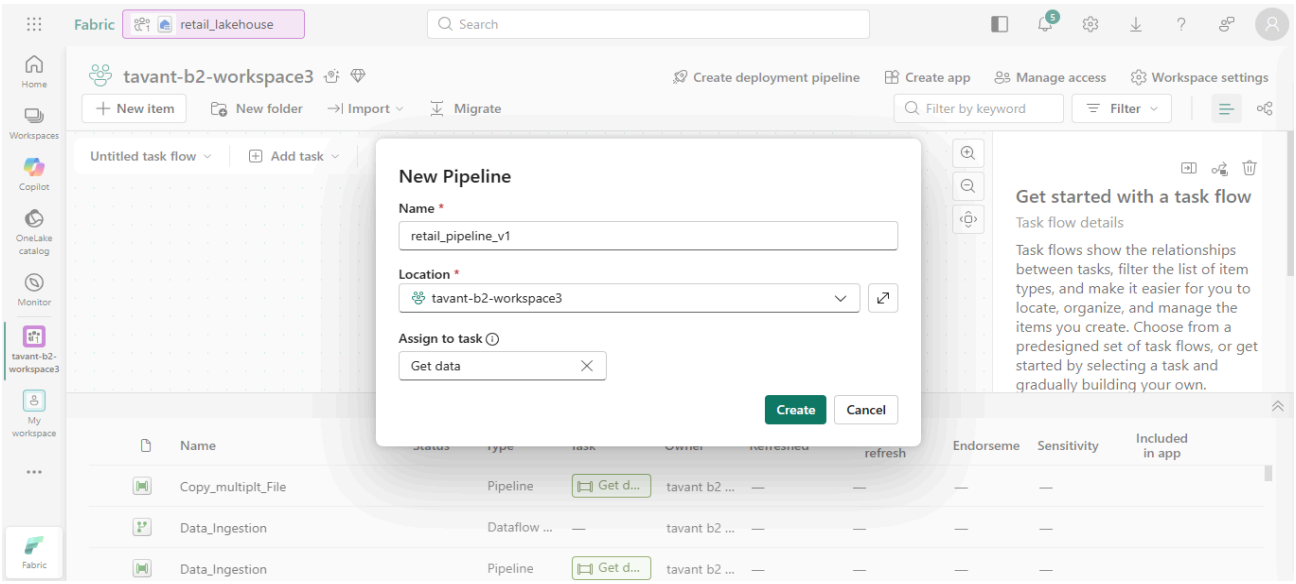
Click:

New Item → Data Pipeline

Name:

retail_pipeline_v1

Click Create.



Expected Output

Pipeline canvas opens.

Top ribbon shows:

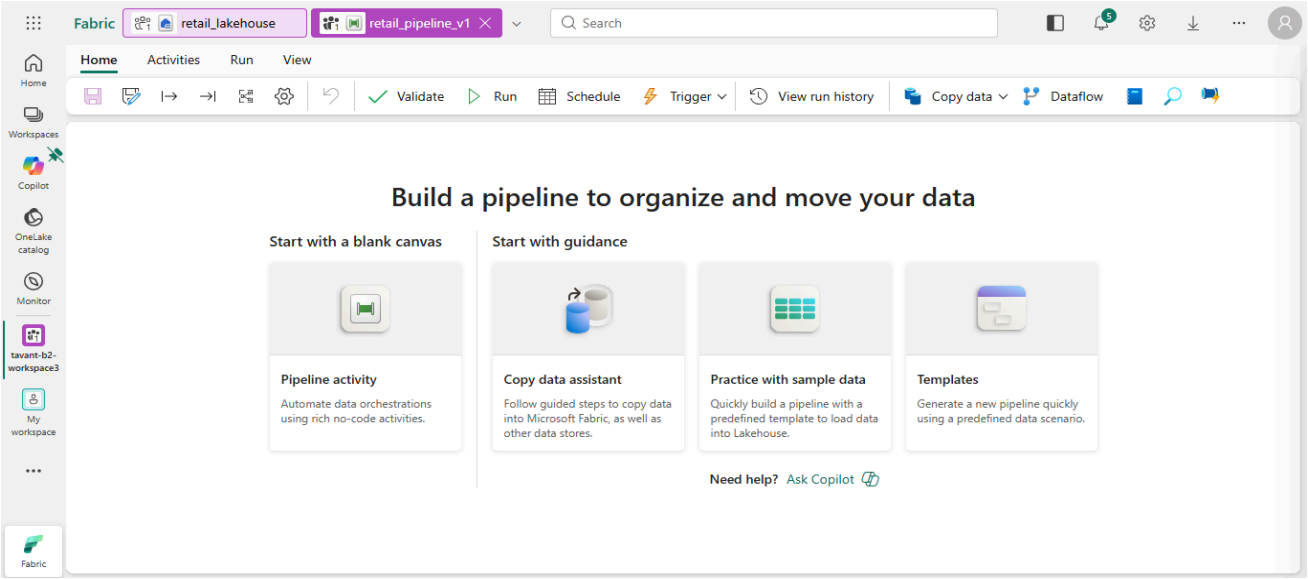
Copy data

Dataflow

Notebook

Lookup

Get metadata



Step 5 — Drag Copy Data Activity

Actions

Click:

Copy data

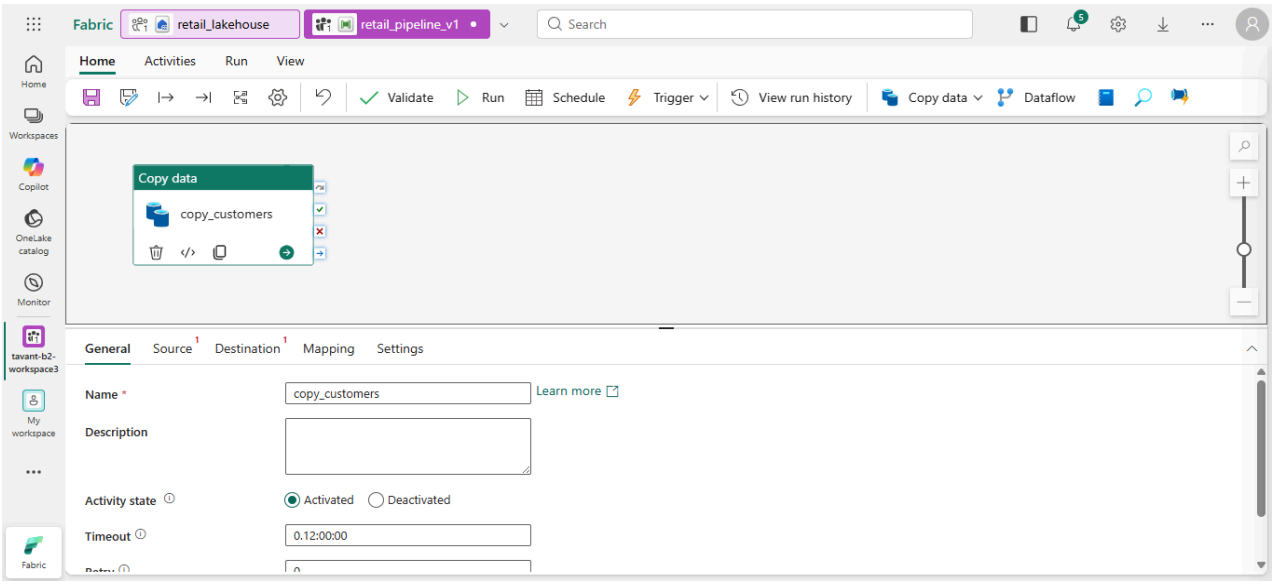
Activity appears on canvas.

Rename activity:

copy_customers

Expected Output

Copy activity visible on pipeline canvas.



Step 6 — Configure Source

Actions

Open:

Source tab

Fill:

Field	Value
Connection	retail_lakehouse

Root Folder	Files
File Path Type	File path

Click:

Browse

Select:

customers.csv

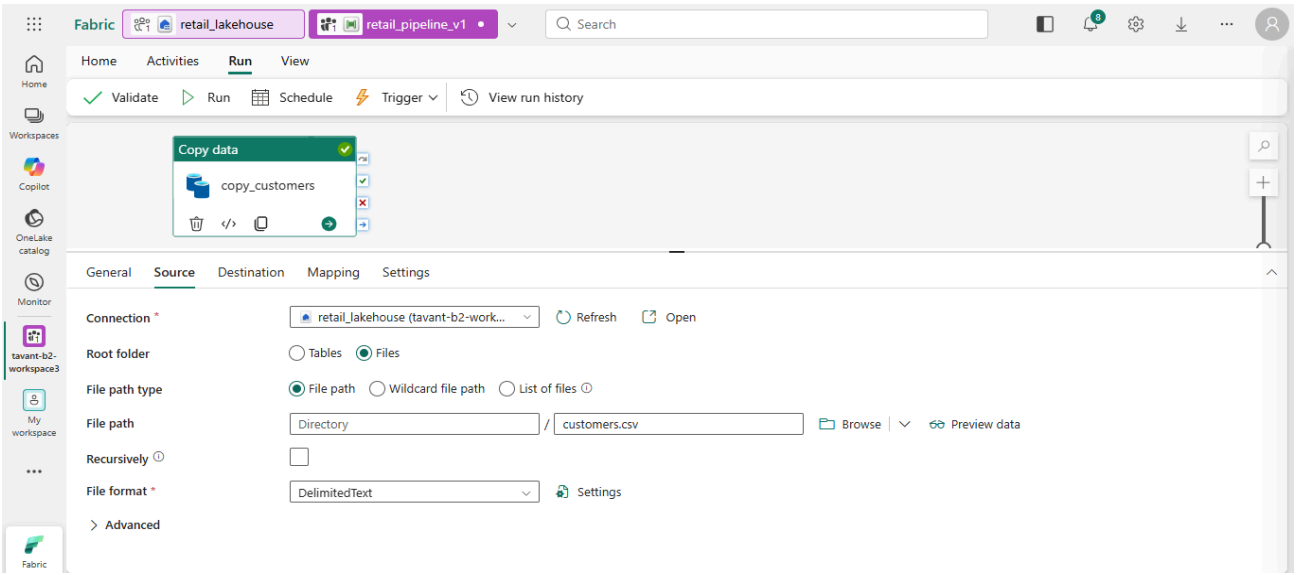
Change File Format

Change:

Binary

TO:

DelimitedText

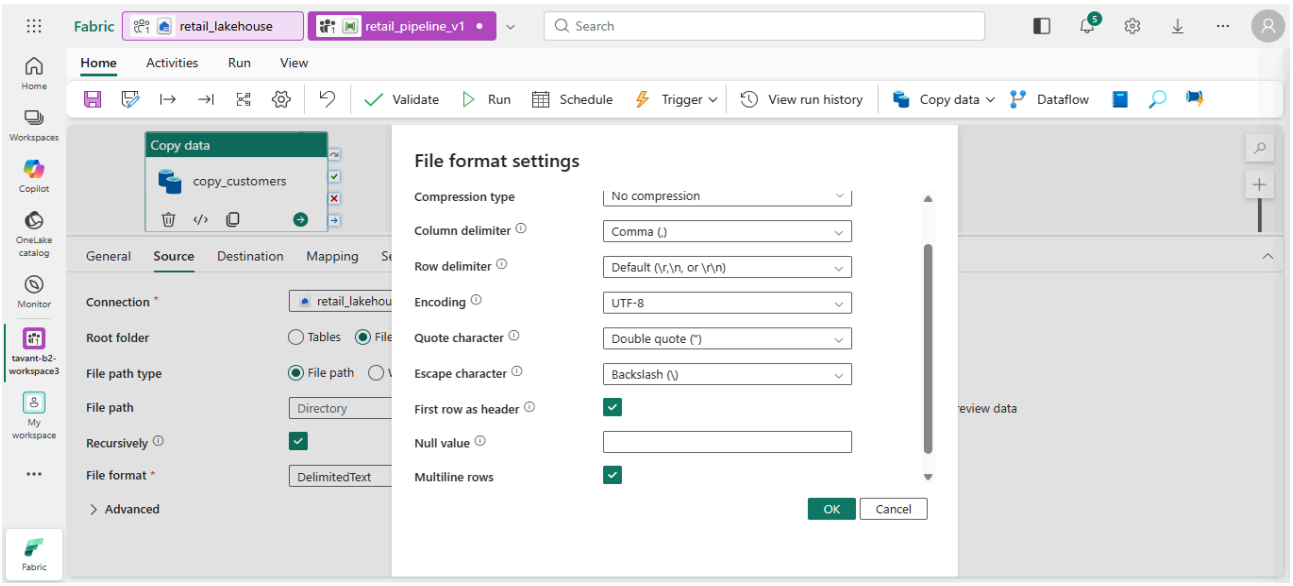


Click:

Settings

Fill:

Setting	Value
Column delimiter	Comma
First row as header	TRUE
Encoding	UTF-8



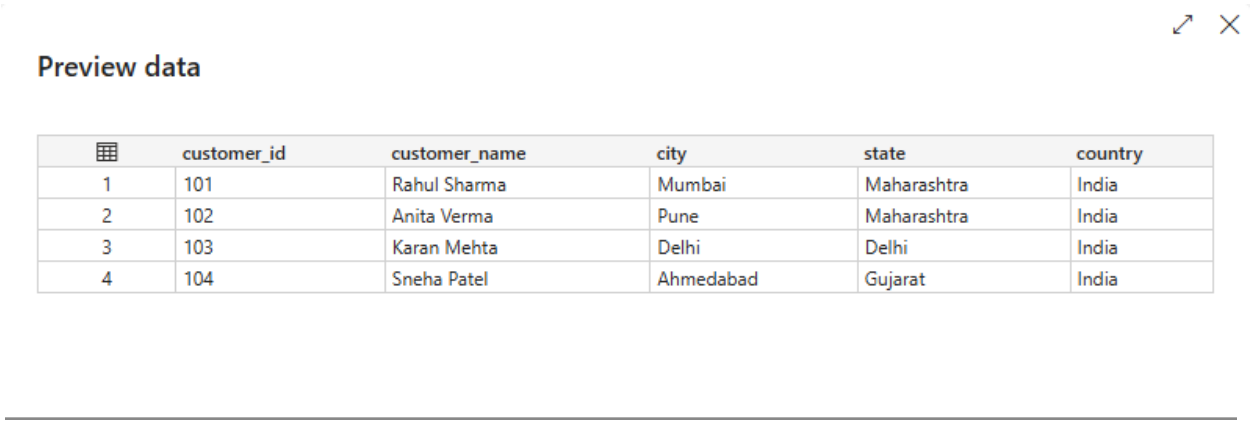
Click:

Preview data

Expected Output

Preview table appears:

customer_id	customer_name	city
101	Rahul Sharma	Mumbai



Preview data

	customer_id	customer_name	city	state	country
1	101	Rahul Sharma	Mumbai	Maharashtra	India
2	102	Anita Verma	Pune	Maharashtra	India
3	103	Karan Mehta	Delhi	Delhi	India
4	104	Sneha Patel	Ahmedabad	Gujarat	India

Step 7 — Configure Destination

Actions

Open:

Destination tab

Fill:

Field	Value
Connection	retail_lakehouse
Root Folder	Tables
Schema name	dbo

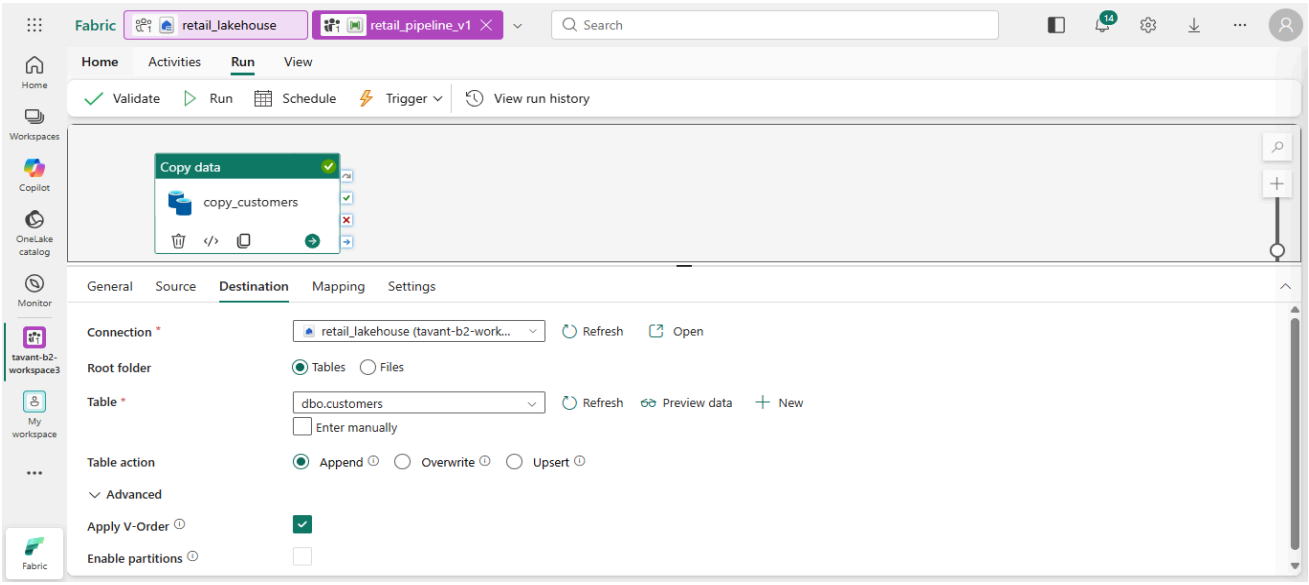
Table	customers
Table Action	Append

Check:

Apply V-Order

Leave unchecked:

Enable partitions



Expected Output

Destination configured as:

dbo.customers

Step 8 — Configure Mapping

Actions

Open:

Mapping tab

Click:

Import Schemas

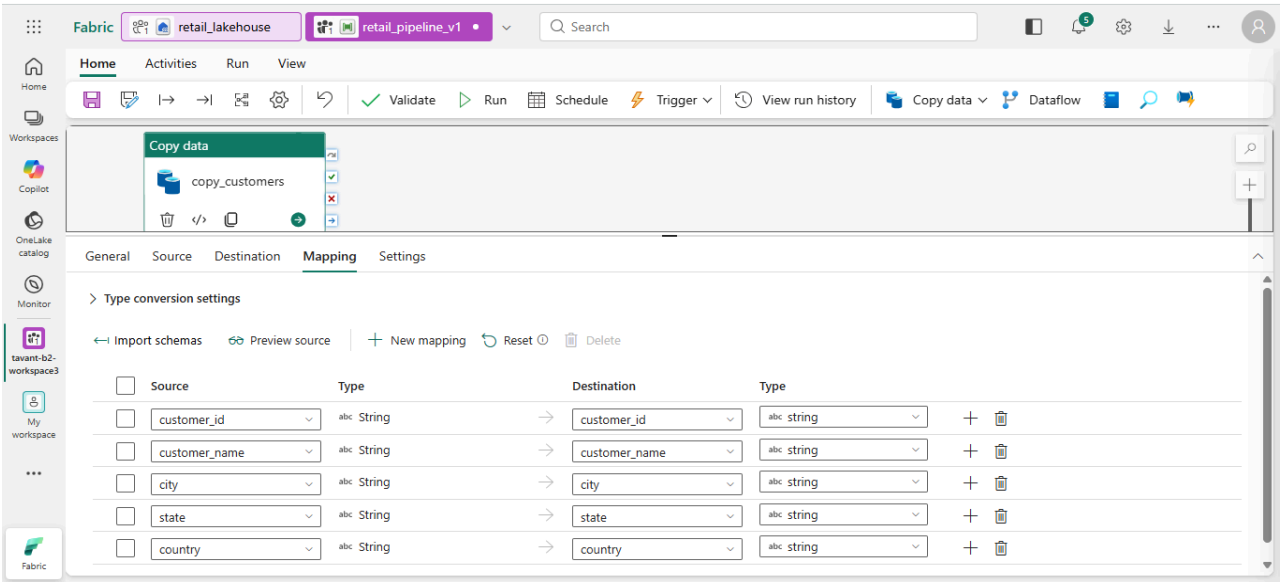
Expected Output

Column mapping appears.

Example:

Source	Destination
customer_id	customer_id

customer_name	customer_name



Step 9 — Configure Pipeline Settings

Actions

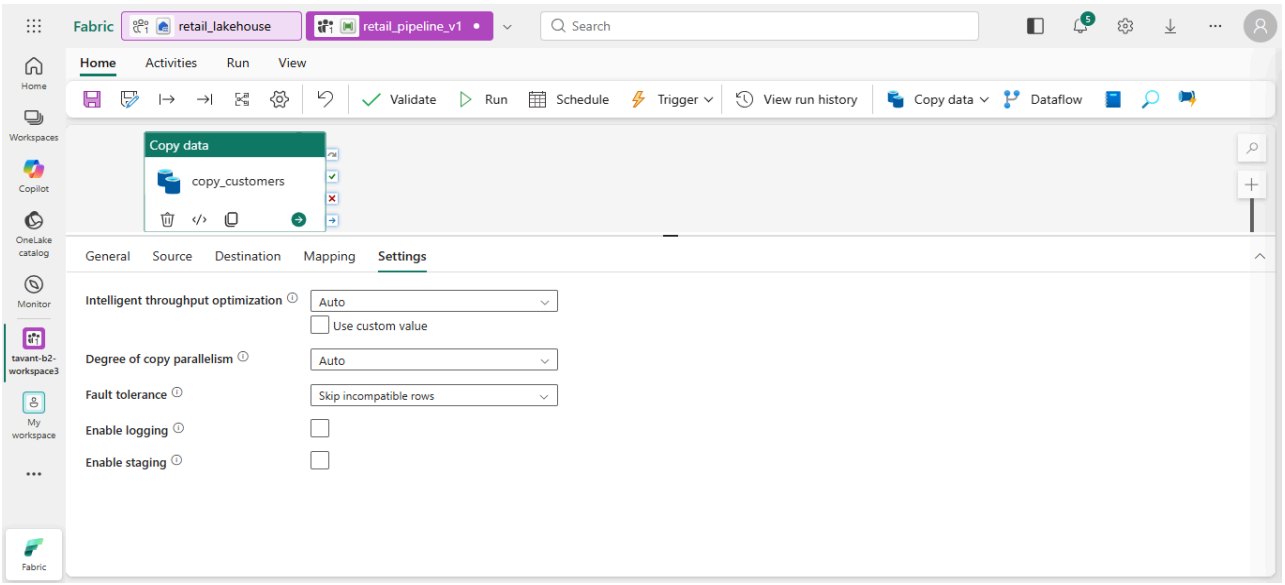
Open:

Settings tab

Fill:

Setting	Value

Intelligent throughput optimization	Auto
Degree of copy parallelism	Auto
Fault tolerance	Skip incompatible rows
Enable logging	OFF



Expected Output

Settings saved successfully.

Step 10 — Execute Pipeline

Actions

Click:

Run

OR

Run pipeline

Expected Output

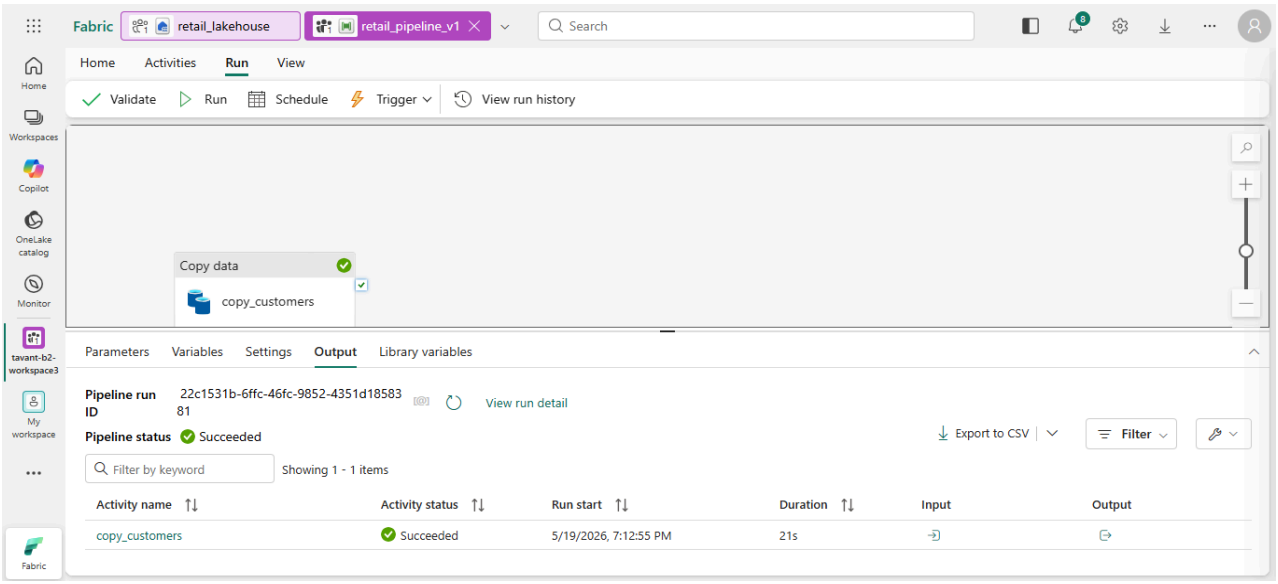
Pipeline status changes:

Queued

→ In Progress

→ Succeeded

Green success icon appears.



Step 11 — Validate Lakehouse Tables

Actions

Return to:

retail_lakehouse

Open:

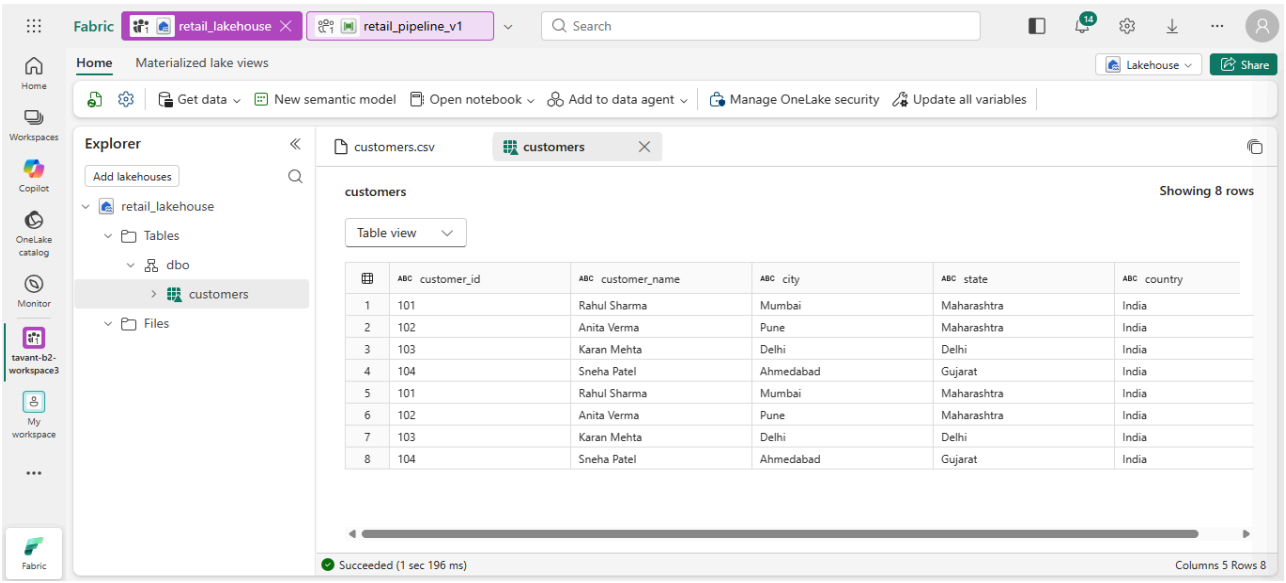
Tables

Expected Output

You should see:

customers

Open table preview.



Step 12 — Repeat for Remaining Files

Repeat Steps 5–11 for:

CSV	Table
products.csv	products
sales.csv	sales

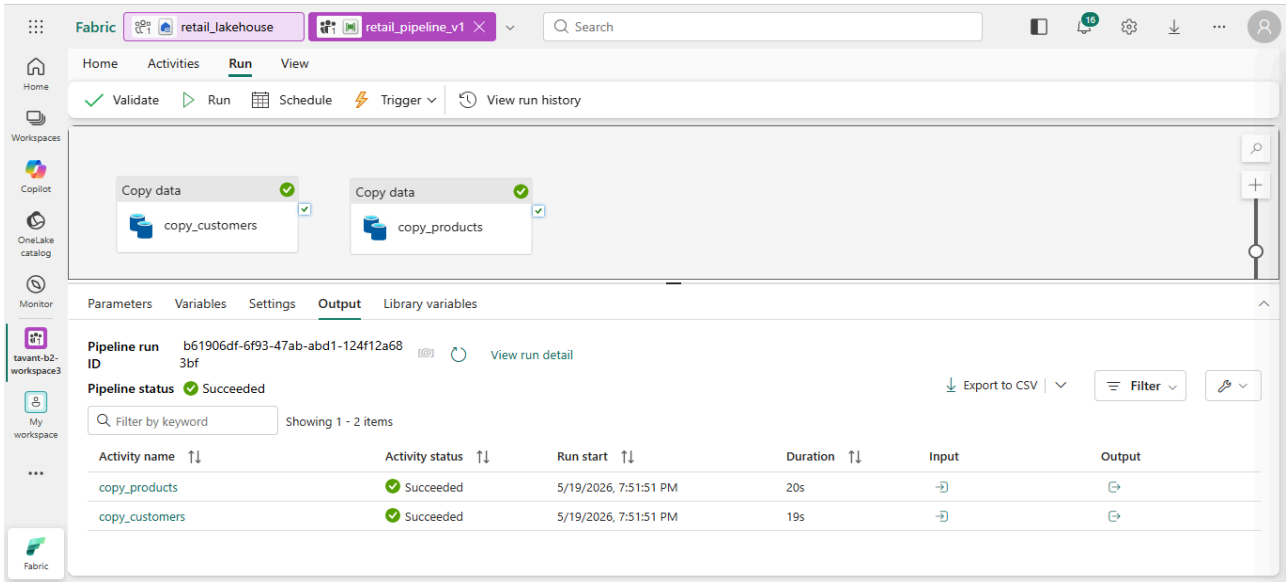
Expected Output

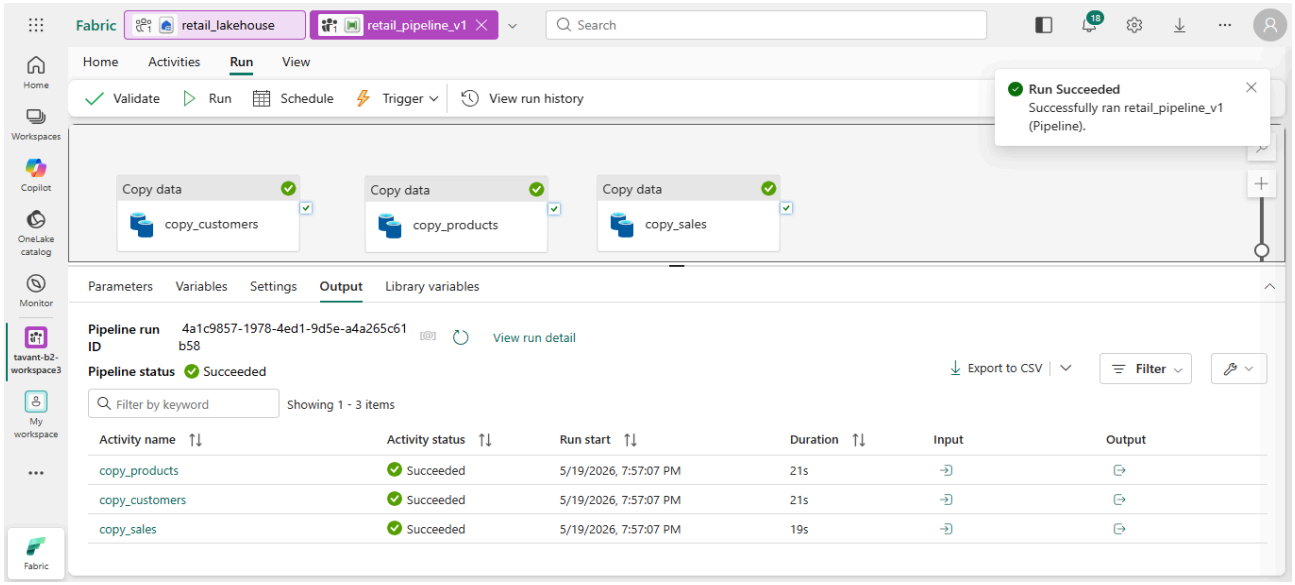
Tables visible:

customers

products

sales





Step 13 — Add Notebook

Actions

Inside pipeline:

Click:

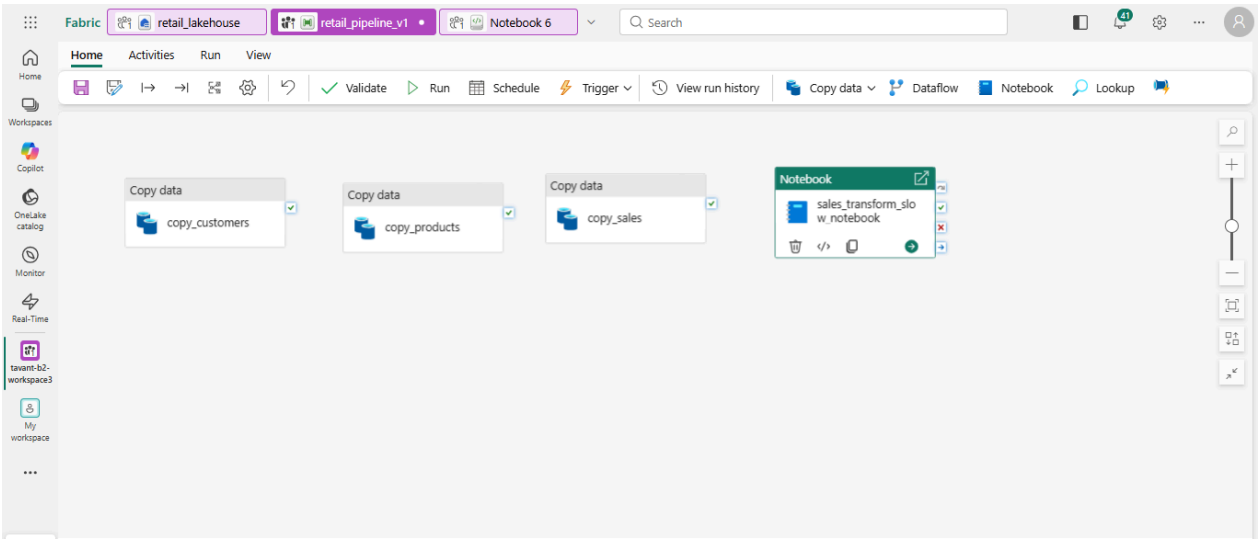
Notebook

Rename:

sales_transform_slow_notebook

Expected Output

Notebook activity added.



Step 14 — Add Inefficient Queries

Perform:

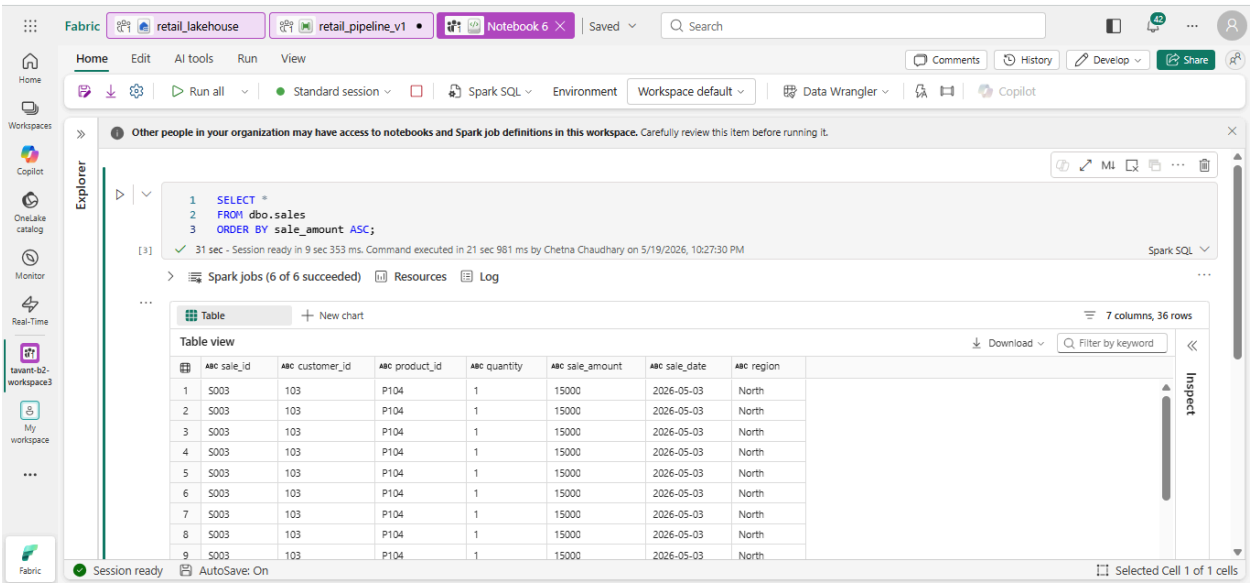
Multiple Sorts:

Run:

SELECT *

FROM dbo.sales

ORDER BY sale_amount ASC;

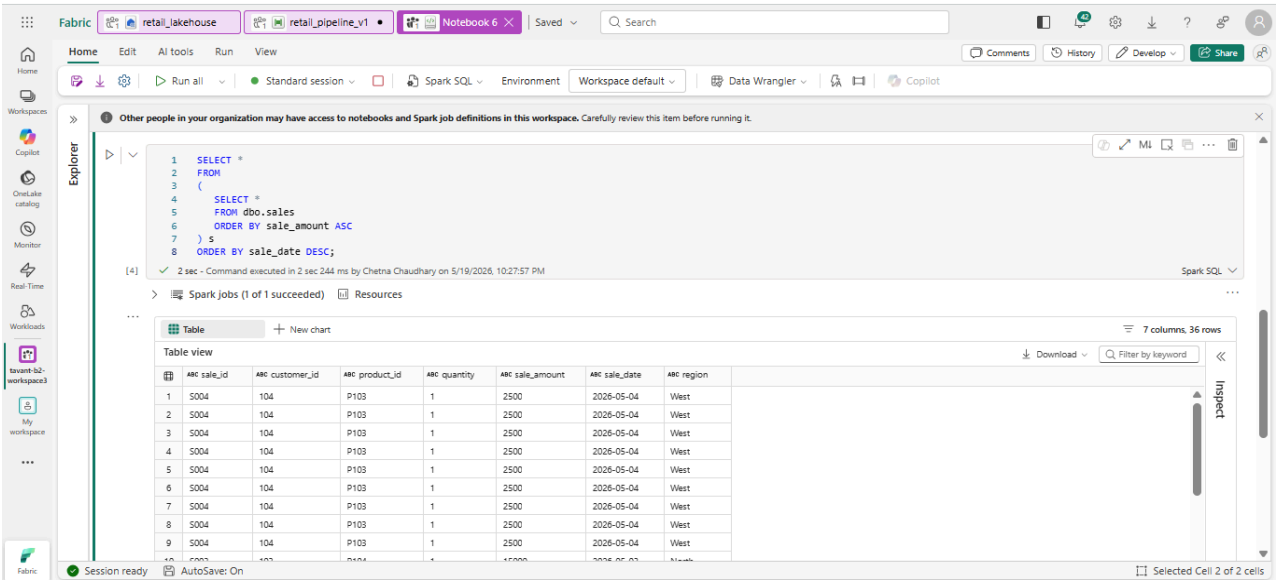


Run Another Sort:

SELECT *

FROM

```
(
  SELECT *
  FROM dbo.sales
  ORDER BY sale_amount ASC
) s
ORDER BY sale_date DESC;
```



Expected Output

Rows returned sorted twice.

Example:

sale_id	sale_amount	sale_date

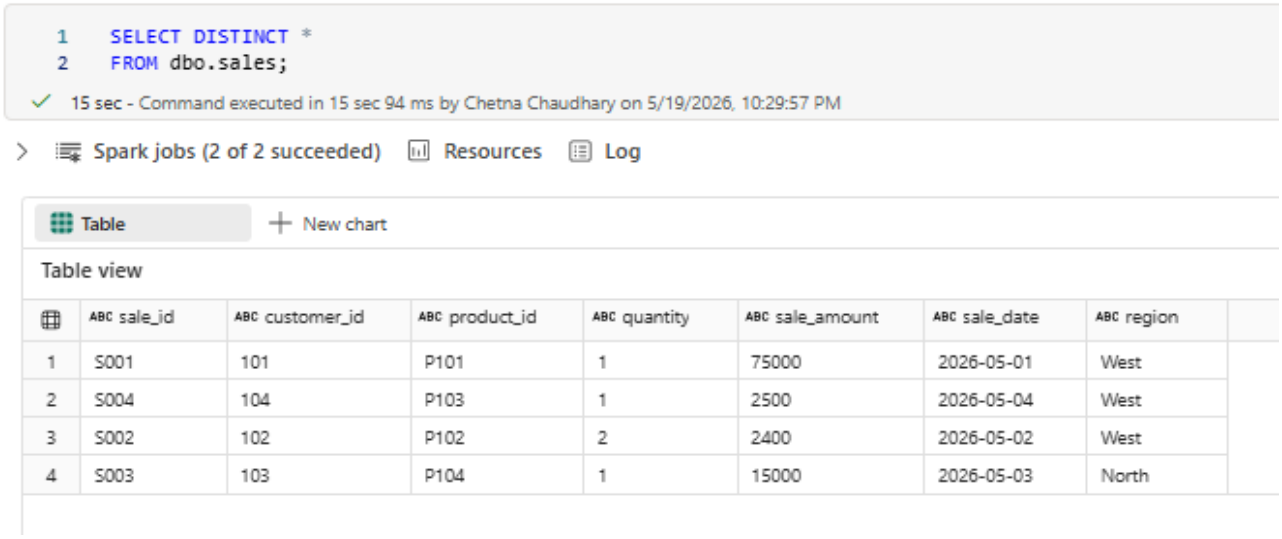
S001	75000	2026-05-01

Duplicate Filtering:

Run:

SELECT DISTINCT *

FROM dbo.sales;



	sale_id	customer_id	product_id	quantity	sale_amount	sale_date	region
1	S001	101	P101	1	75000	2026-05-01	West
2	S004	104	P103	1	2500	2026-05-04	West
3	S002	102	P102	2	2400	2026-05-02	West
4	S003	103	P104	1	15000	2026-05-03	North

More Expensive Deduplication:

SELECT DISTINCT

sale_id,

customer_id,

product_id,

quantity,

sale_amount,

sale_date,

region

FROM dbo.sales;

```
SELECT DISTINCT
sale_id,
customer_id,
product_id,
quantity,
sale_amount,
sale_date,
region
FROM dbo.sales;
```

3 sec - Command executed in 3 sec 224 ms by Chetna Chaudhary on 5/19/2026, 10:30:26 PM

Spark jobs (2 of 2 succeeded) Resources Log

Table + New chart

Table view

	sale_id	customer_id	product_id	quantity	sale_amount	sale_date	region
1	S001	101	P101	1	75000	2026-05-01	West
2	S004	104	P103	1	2500	2026-05-04	West
3	S002	102	P102	2	2400	2026-05-02	West
4	S003	103	P104	1	15000	2026-05-03	North

Expected Output

Unique rows returned.

Repeated Joins:

Run:

SELECT

s.sale_id,

s.sale_amount,

c.customer_name,

c.city

FROM dbo.sales s

INNER JOIN dbo.customers c

ON s.customer_id = c.customer_id;

```
1   SELECT
2   s.sale_id,
3   s.sale_amount,
4   c.customer_name,
5   c.city
6   FROM dbo.sales s
7   INNER JOIN dbo.customers c
8   ON s.customer_id = c.customer_id;
```

✓ 13 sec - Command executed in 13 sec 501 ms by Chetna Chaudhary on 5/19/2026, 10:31:17 PM

> Spark jobs (11 of 11 succeeded) Resources Log

Table + New chart

Table view

	sale_id	sale_amount	customer_name	city
1	S001	75000	Rahul Sharma	Mumbai
2	S001	75000	Rahul Sharma	Mumbai
3	S001	75000	Rahul Sharma	Mumbai
4	S001	75000	Rahul Sharma	Mumbai
5	S001	75000	Rahul Sharma	Mumbai
6	S001	75000	Rahul Sharma	Mumbai
7	S001	75000	Rahul Sharma	Mumbai
8	S001	75000	Rahul Sharma	Mumbai
9	S001	75000	Rahul Sharma	Mumbai
10	S002	2400	Anita Verma	Pune

Add Another Join:

```
SELECT

s.sale_id,

s.sale_amount,

c.customer_name,

p.product_name,

p.category

FROM dbo.sales s

INNER JOIN dbo.customers c

ON s.customer_id = c.customer_id

INNER JOIN dbo.products p

ON s.product_id = p.product_id;
```

```sql
SELECT
s.sale_id,
s.sale_amount,
c.customer_name,
p.product_name,
p.category
FROM dbo.sales s
INNER JOIN dbo.customers c
ON s.customer_id = c.customer_id
INNER JOIN dbo.products p
ON s.product_id = p.product_id;
```

9 sec - Command executed in 9 sec 390 ms by Chetna Chaudhary on 5/19/2026, 10:31:55 PM

Spark jobs (10 of 10 succeeded) Resources

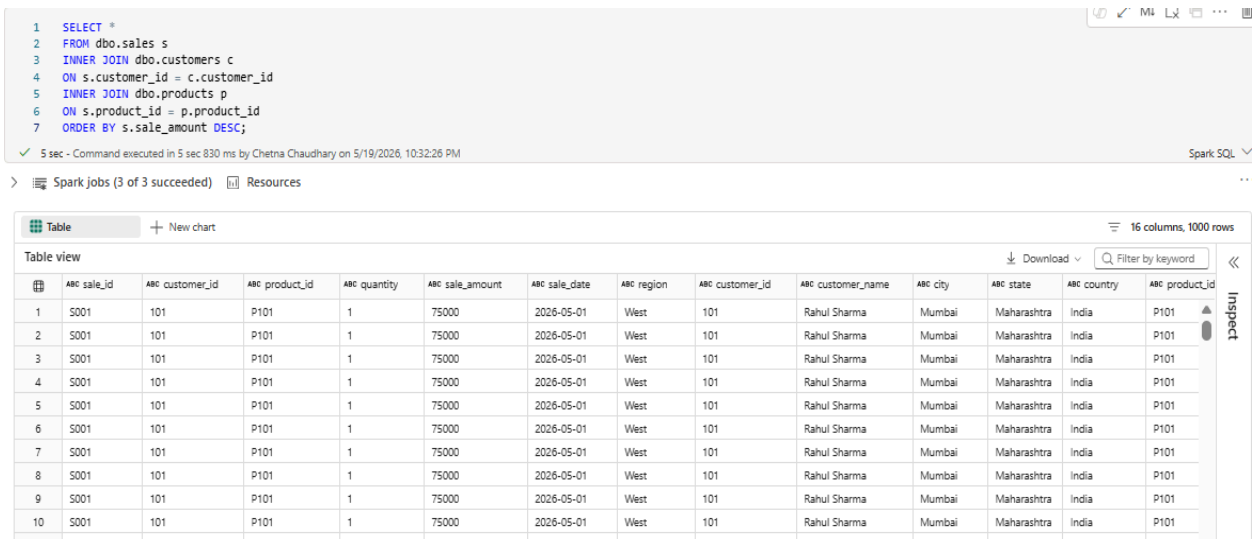
Table New chart

Table view

	sale_id	sale_amount	customer_name	product_name	category
1	S001	75000	Rahul Sharma	Laptop	Electronics
2	S001	75000	Rahul Sharma	Laptop	Electronics
3	S001	75000	Rahul Sharma	Laptop	Electronics
4	S001	75000	Rahul Sharma	Laptop	Electronics
5	S001	75000	Rahul Sharma	Laptop	Electronics
6	S001	75000	Rahul Sharma	Laptop	Electronics
7	S001	75000	Rahul Sharma	Laptop	Electronics
8	S001	75000	Rahul Sharma	Laptop	Electronics

Even More Inefficient Version:

```
SELECT *

FROM dbo.sales s

INNER JOIN dbo.customers c

ON s.customer_id = c.customer_id

INNER JOIN dbo.products p

ON s.product_id = p.product_id

ORDER BY s.sale_amount DESC;
```



```
SELECT *
FROM dbo.sales s
INNER JOIN dbo.customers c
ON s.customer_id = c.customer_id
INNER JOIN dbo.products p
ON s.product_id = p.product_id
ORDER BY s.sale_amount DESC;
```

5 sec - Command executed in 5 sec 830 ms by Chetna Chaudhary on 5/19/2026, 10:32:26 PM — Spark SQL

Spark jobs (3 of 3 succeeded) Resources

Table | New chart | 16 columns, 1000 rows

Table view | Download | Filter by keyword | Inspect

	sale_id	customer_id	product_id	quantity	sale_amount	sale_date	region	customer_id	customer_name	city	state	country	product_id
1	S001	101	P101	1	75000	2026-05-01	West	101	Rahul Sharma	Mumbai	Maharashtra	India	P101
2	S001	101	P101	1	75000	2026-05-01	West	101	Rahul Sharma	Mumbai	Maharashtra	India	P101
3	S001	101	P101	1	75000	2026-05-01	West	101	Rahul Sharma	Mumbai	Maharashtra	India	P101
4	S001	101	P101	1	75000	2026-05-01	West	101	Rahul Sharma	Mumbai	Maharashtra	India	P101
5	S001	101	P101	1	75000	2026-05-01	West	101	Rahul Sharma	Mumbai	Maharashtra	India	P101
6	S001	101	P101	1	75000	2026-05-01	West	101	Rahul Sharma	Mumbai	Maharashtra	India	P101
7	S001	101	P101	1	75000	2026-05-01	West	101	Rahul Sharma	Mumbai	Maharashtra	India	P101
8	S001	101	P101	1	75000	2026-05-01	West	101	Rahul Sharma	Mumbai	Maharashtra	India	P101
9	S001	101	P101	1	75000	2026-05-01	West	101	Rahul Sharma	Mumbai	Maharashtra	India	P101
10	S001	101	P101	1	75000	2026-05-01	West	101	Rahul Sharma	Mumbai	Maharashtra	India	P101

Expected Output

Joined dataset returned:

sale_id	customer_name	product_name
S001	Rahul Sharma	Laptop

Step 15 — Open Monitor Hub & Monitor Slow Pipeline

Actions

From left panel:

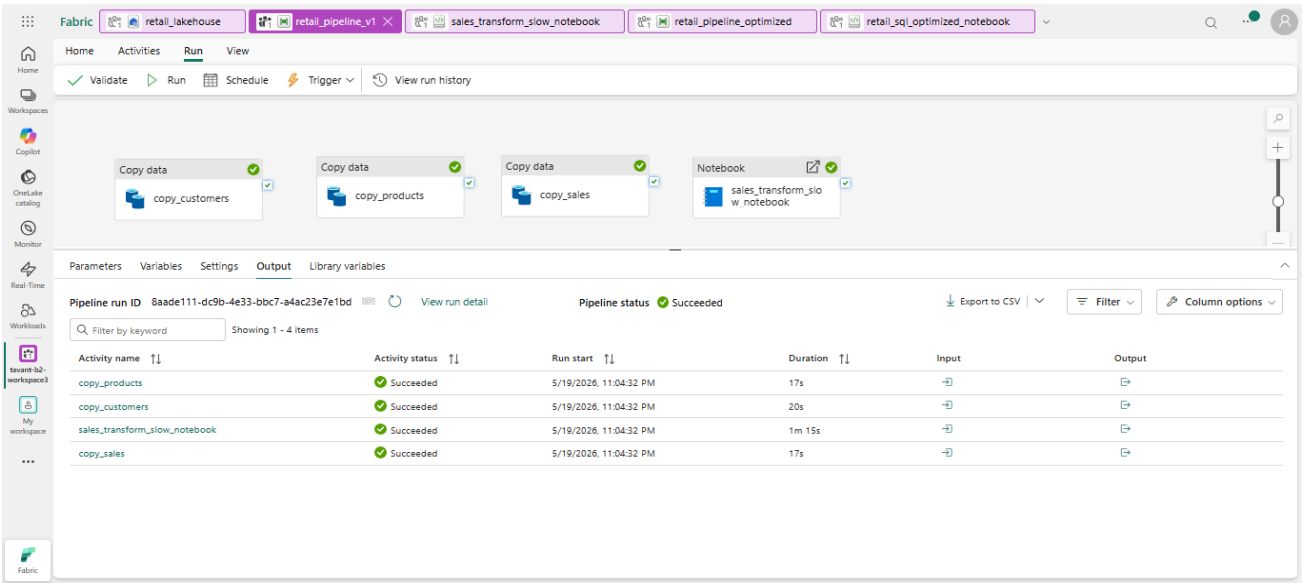
Monitor

Open pipeline run.

Expected Output

You should see:

Metric	Example
Runtime	Higher
Queue time	Increased
Copy duration	Slow



Step 16 — Duplicate Pipeline

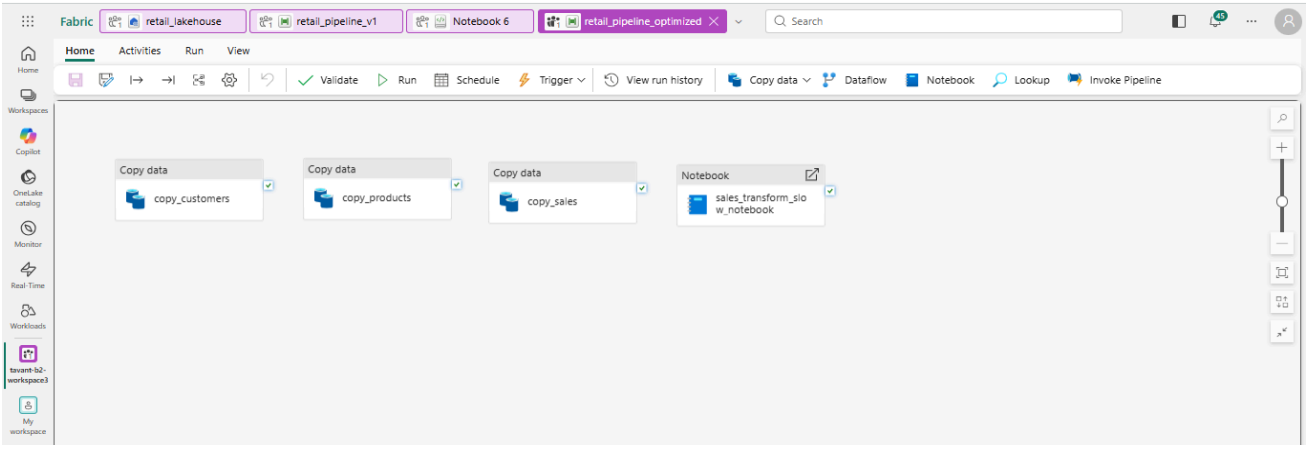
Actions

Duplicate:

retail_pipeline_v1

Rename:

retail_pipeline_optimized



Expected Output

Optimized copy created.

Step 17 — Improve Performance

Actions

Inside workspace locate:

sales_transform_slow_notebook

Click:

··· → Duplicate

Rename:

retail_sql_optimized_notebook

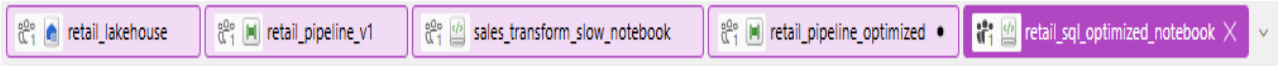
Set:

Parallel Copies = 4

Expected Output

You now have:

Notebook
sales_transform_slow_notebook
retail_sql_optimized_notebook



Open Optimized Notebook:

Actions

Open:

retail_sql_optimized_notebook

Expected Output

Notebook editor opens successfully.

Remove Repeated Sorts:

OLD SLOW QUERY

Remove this:

```
SELECT *
FROM
(
  SELECT *
  FROM dbo.sales
  ORDER BY sale_amount ASC
) s
ORDER BY sale_date DESC;
```

REPLACE WITH OPTIMIZED QUERY:

SELECT

sale_id,

customer_id,

sale_amount,

sale_date

FROM dbo.sales;

```
1   SELECT
2   sale_id,
3   customer_id,
4   sale_amount,
5   sale_date
6   FROM dbo.sales;
```

✓ 3 sec - Command executed in 3 sec 178 ms by Chetna Chaudhary on 5/19/2026, 10:53:59 PM

> Spark jobs (3 of 3 succeeded) Resources

Table + New chart

Table view

	ABC sale_id	ABC customer_id	ABC sale_amount	ABC sale_date
1	S001	101	75000	2026-05-01
2	S002	102	2400	2026-05-02
3	S003	103	15000	2026-05-03
4	S004	104	2500	2026-05-04
5	S001	101	75000	2026-05-01
6	S002	102	2400	2026-05-02
7	S003	103	15000	2026-05-03
8	S004	104	2500	2026-05-04
9	S001	101	75000	2026-05-01
10	S002	102	2400	2026-05-02
11	S003	103	15000	2026-05-03

Expected Output

Rows returned instantly with lower execution time.

Apply Early Filtering:

OLD QUERY:

SELECT *

FROM sales;

OPTIMIZED QUERY:

SELECT *

FROM sales

WHERE sale_date >= '2026-05-01';

```
1   SELECT *
2   FROM sales
3   WHERE sale_date >= '2026-05-01';
```

✓ 15 sec - Command executed in 15 sec 319 ms by Chetna Chaudhary on 5/19/2026, 10:57:06 PM

> Spark jobs (3 of 3 succeeded) Resources Log

Table + New chart

Table view

	sale_id	customer_id	product_id	quantity	sale_amount	sale_date	region
1	S001	101	P101	1	75000	2026-05-01	West
2	S002	102	P102	2	2400	2026-05-02	West
3	S003	103	P104	1	15000	2026-05-03	North
4	S004	104	P103	1	2500	2026-05-04	West
5	S001	101	P101	1	75000	2026-05-01	West
6	S002	102	P102	2	2400	2026-05-02	West
7	S003	103	P104	1	15000	2026-05-03	North
8	S004	104	P103	1	2500	2026-05-04	West
9	S001	101	P101	1	75000	2026-05-01	West
10	S002	102	P102	2	2400	2026-05-02	West
11	S003	103	P104	1	15000	2026-05-03	North
12	S004	104	P103	1	2500	2026-05-04	West
13	S001	101	P101	1	75000	2026-05-01	West

Expected Output

Smaller filtered dataset returned.

Reduce Joins:

OLD HEAVY JOIN

SELECT *

FROM sales s

INNER JOIN customers c

ON s.customer_id = c.customer_id

INNER JOIN products p

ON s.product_id = p.product_id;

OPTIMIZED JOIN

SELECT

s.sale_id,

s.sale_amount,

p.product_name

FROM sales s

INNER JOIN products p

ON s.product_id = p.product_id;

```
1  SELECT
2  s.sale_id,
3  s.sale_amount,
4  p.product_name
5  FROM sales s
6  INNER JOIN products p
7  ON s.product_id = p.product_id;
8
```

✓ 10 sec - Command executed in 9 sec 994 ms by Chetna Chaudhary on 5/19/2026, 11:00:11 PM

Spark jobs (10 of 10 succeeded) Resources Log

Table + New chart

Table view

	ABC sale_id	ABC sale_amount	ABC product_name
1	S001	75000	Laptop
2	S001	75000	Laptop
3	S001	75000	Laptop
4	S001	75000	Laptop
5	S001	75000	Laptop
6	S001	75000	Laptop
7	S001	75000	Laptop
8	S001	75000	Laptop
9	S001	75000	Laptop
10	S001	75000	Laptop

Expected Output

Smaller joined dataset.

Create Partitioned Table

Add New SQL Cell

CREATE OR REPLACE TABLE sales_partitioned

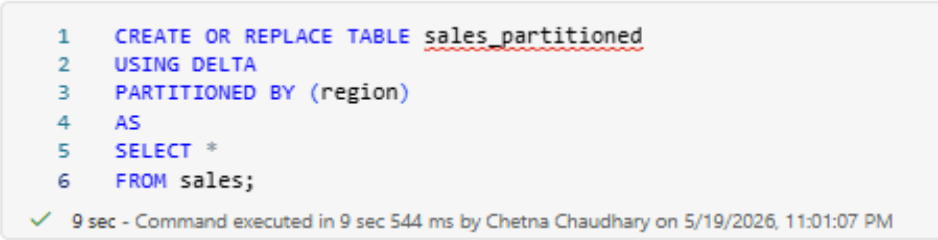
USING DELTA

PARTITIONED BY (region)

AS

SELECT *

FROM sales;



Expected Output

New table created:

sales_partitioned

Create Aggregated Summary Table

Add SQL Cell

CREATE OR REPLACE TABLE sales_summary AS

SELECT

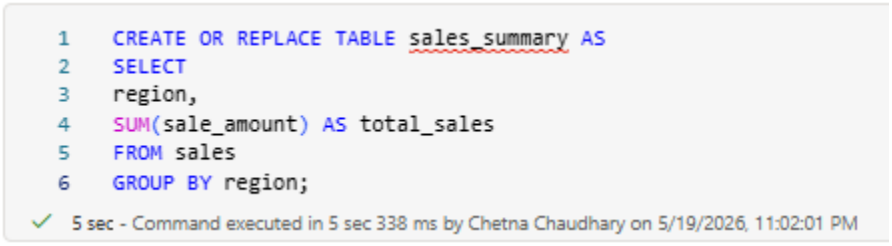
---

region,

SUM(sale_amount) AS total_sales

FROM sales

GROUP BY region;



Expected Output

Summary table created.

Enable Parallelism in Pipeline

Actions

Click:

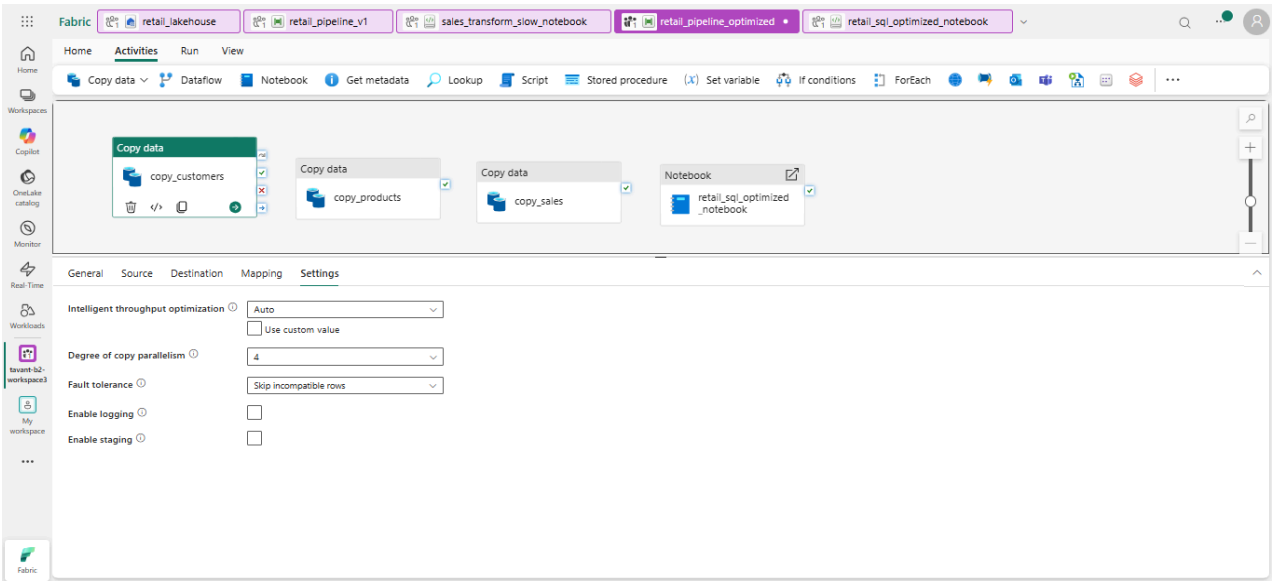
- Copy activity
- Settings tab

Set

Setting	Value
Intelligent throughput optimization	Auto
Degree of copy parallelism	4

Expected Output

Pipeline copy activity configured for parallel execution.



Step 18 — Run Optimized Pipeline

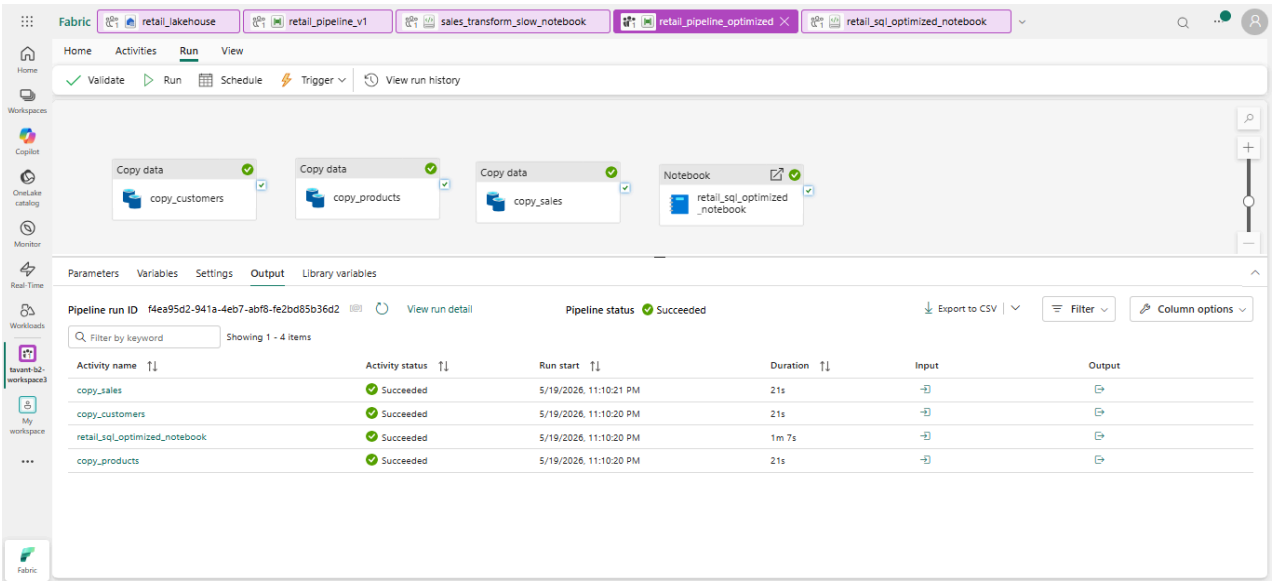
Actions

Run optimized pipeline.

Compare with old pipeline.

Expected Output

Metric	Old	New
Runtime	5 min	2 min
CPU	High	Lower
Throughput	Slow	Faster



Validation

Performance improved successfully.

Step 19 — Governance Policies

Implement Workspace Governance (RBAC)

Actions

Open:

Workspaces → tavant-b2-workspace3

Click:

Manage Access

Add Users / Roles

Assign roles like:

Role	Permission	Governance Purpose
Viewer	Read-only	Controlled data access
Contributor	Edit limited assets	Managed development
Member	Full workspace editing	Team collaboration
Admin	Full control	Governance administration

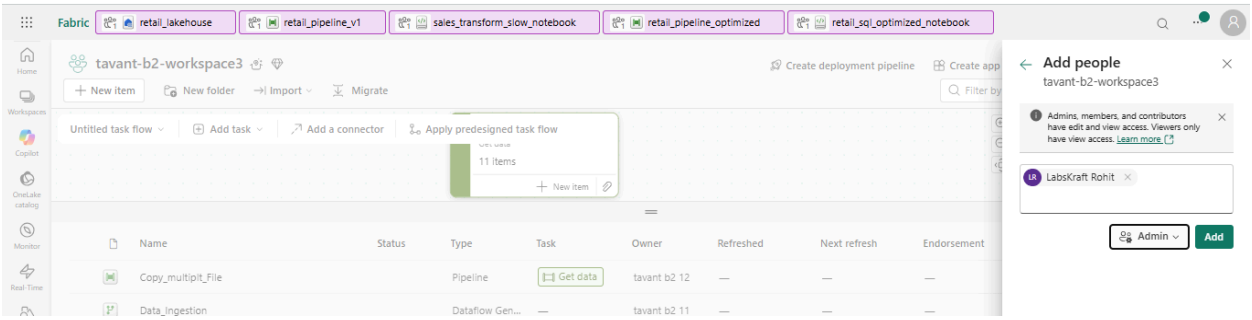
Expected Output

Users appear under workspace access list.

Example:

User	Role
analyst_user	Viewer

engineer_user	Contributor



Governance Validation

- Role-based governance implemented
- Least-privilege access enforced

Governance Through Data Quality Validation

Open Notebook

Open:

retail_sql_optimized_notebook

Add SQL Validation Cell

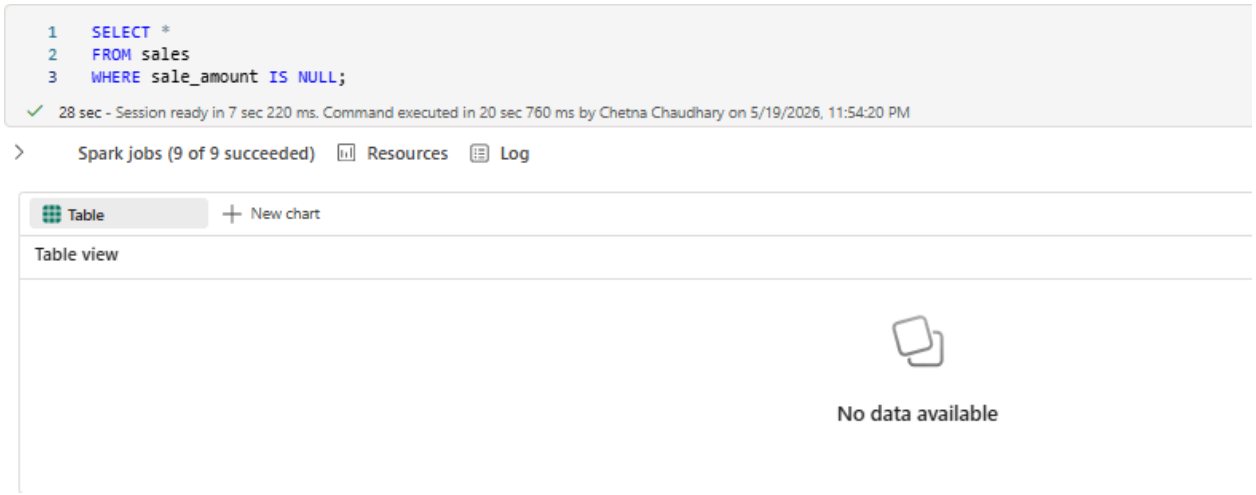
SELECT *

FROM sales

WHERE sale_amount IS NULL;

Expected Output

No NULL rows returned.



Governance Validation

- Data quality governance implemented

Governance Through Monitoring & Audit

Actions

From left panel open:

Monitor

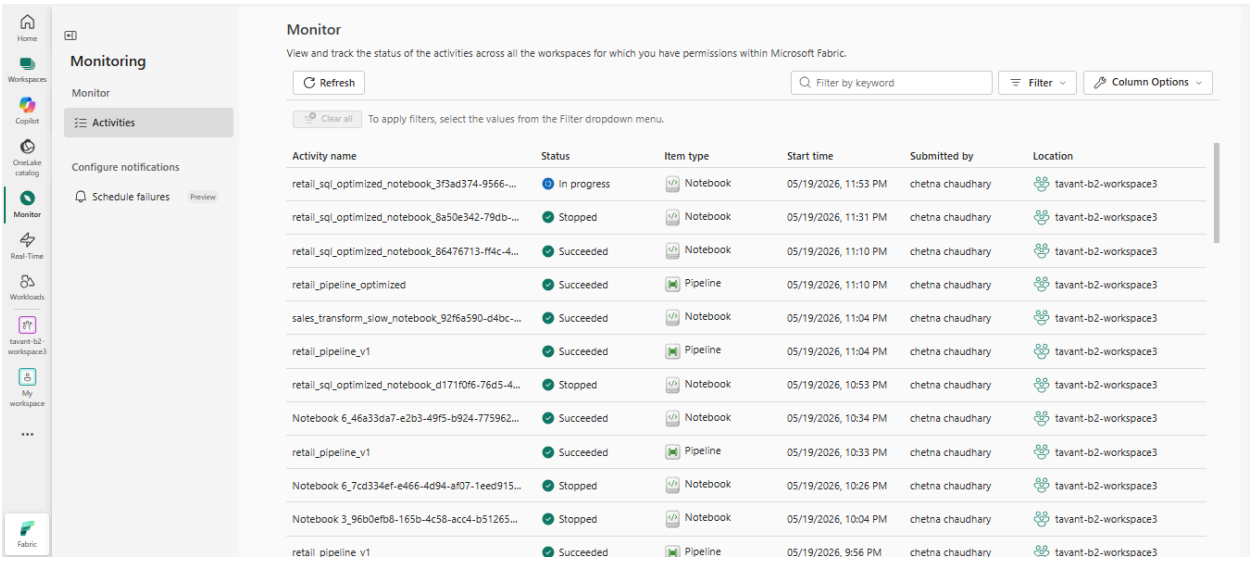
Observe

You should see:

- pipeline runs
- notebook runs
- copy activity execution
- success/failure status
- execution duration

Expected Output

Activity	Status
copy_sales	Succeeded
notebook	Succeeded



Governance Validation

- Audit monitoring enabled
- Operational governance implemented

Step 20 — Create Warehouse

Actions

Inside:

tavant-b2-workspace3

Click:

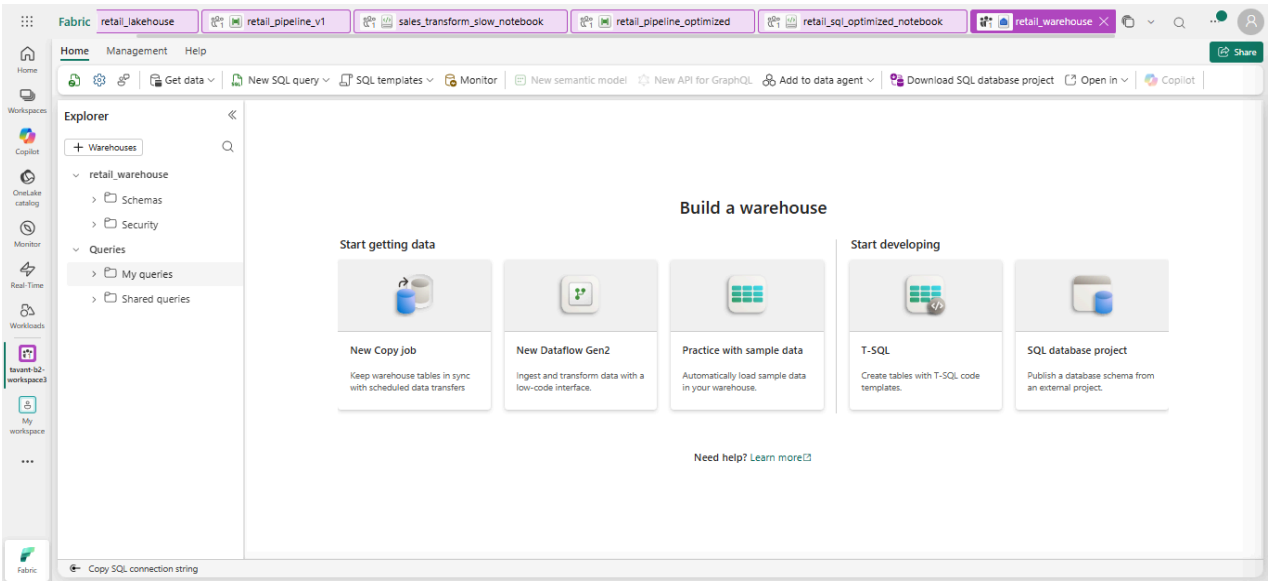
New Item → Warehouse

Name:

retail_warehouse

Expected Output

Warehouse SQL editor opens.



Step 21 — Create Security Table

Run:

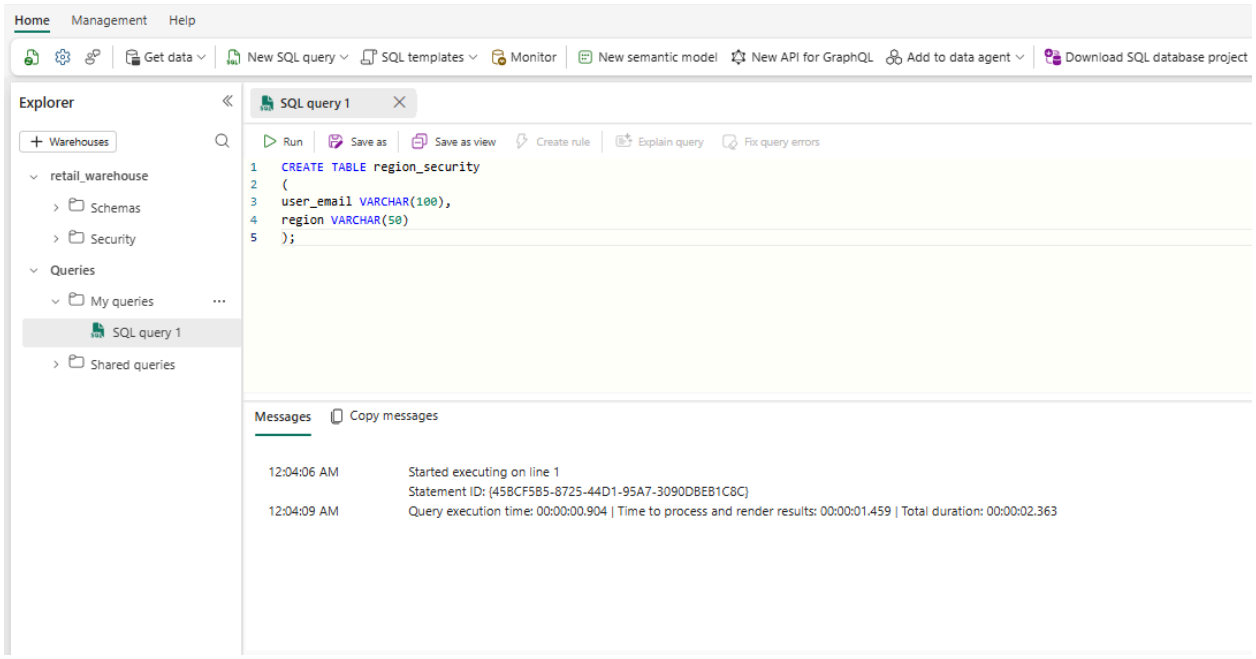
CREATE TABLE region_security

(

user_email VARCHAR(100),

region VARCHAR(50)

);



Expected Output

Command completed successfully

Step 22 — Insert Security Data

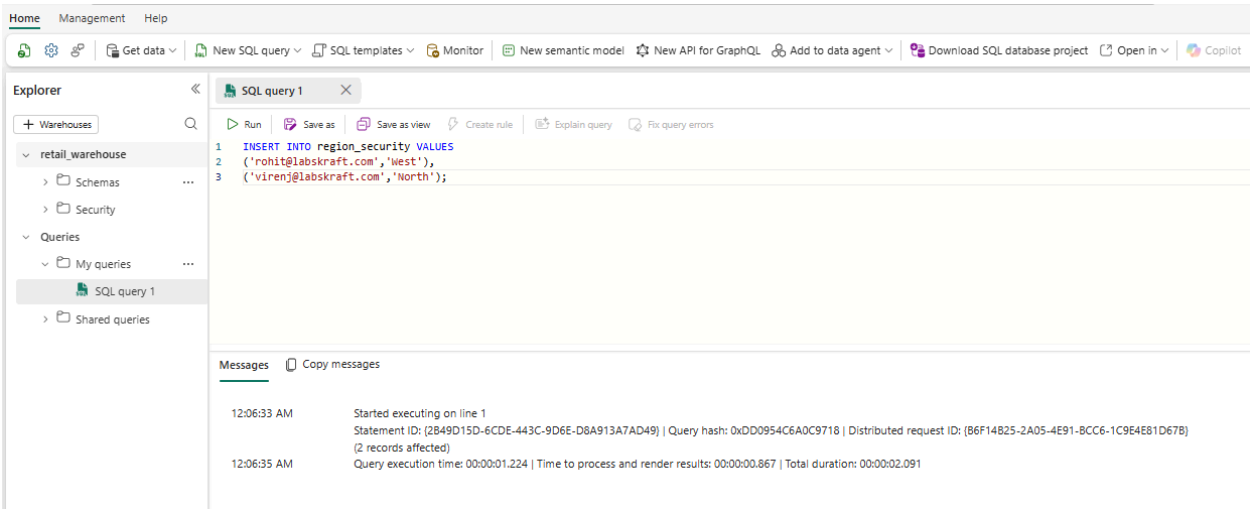
INSERT INTO region_security VALUES

('west_user@company.com','West'),

('north_user@company.com','North');

Expected Output

Rows inserted successfully.



Step 23 — Create RLS Policy

Run:

CREATE FUNCTION dbo.fn_region_access(@region AS VARCHAR(50))

RETURNS TABLE

WITH SCHEMABINDING

AS

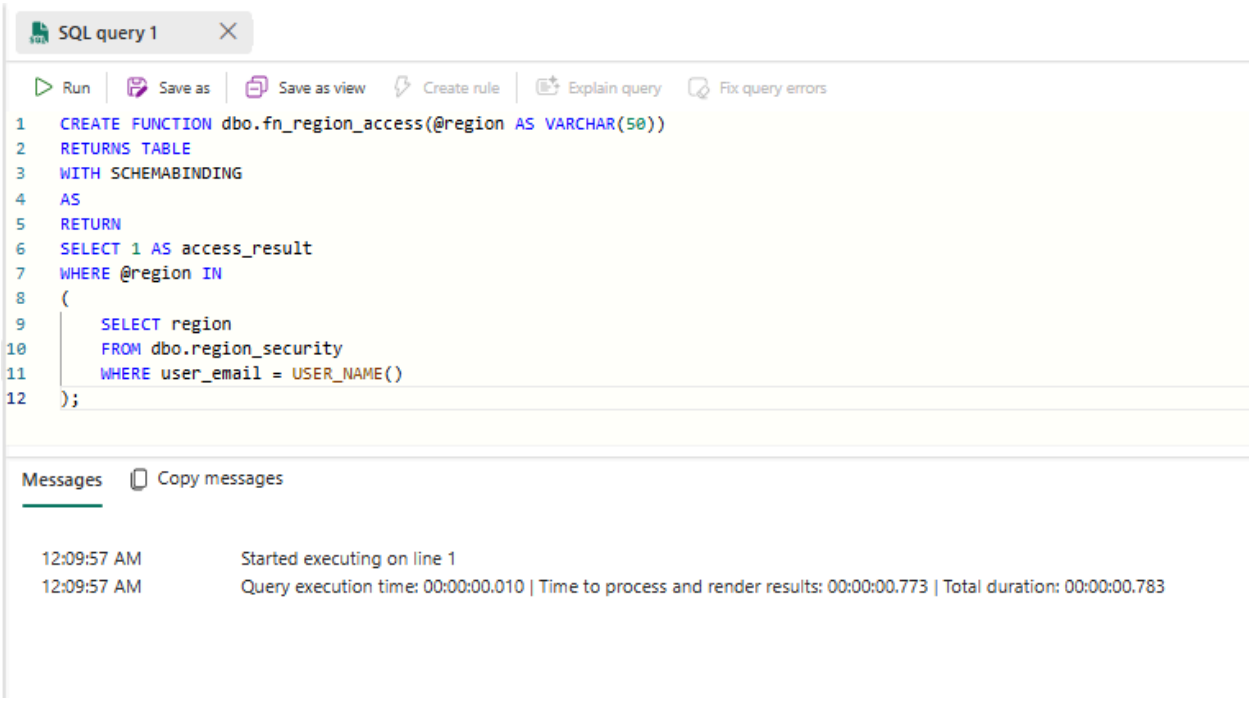
RETURN

SELECT 1 AS access_result

WHERE @region IN

(

```
    SELECT region
    FROM dbo.region_security
    WHERE user_email = USER_NAME()
);
```



Expected Output

Function created successfully.

Step 24 — Load data from lakehouse to warehouse

Use Data Pipeline

Open:

retail_pipeline_optimized

Add Copy Activity

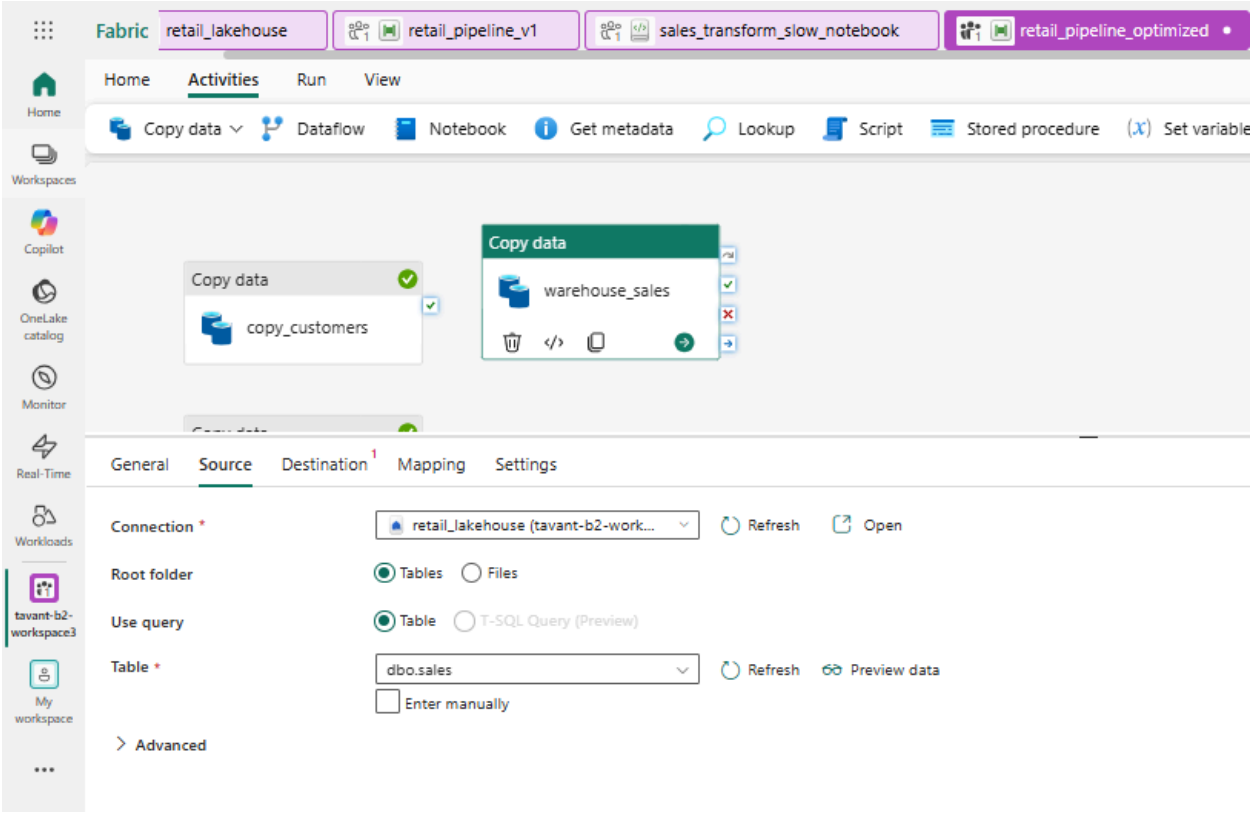
Drag:

Copy Data

onto pipeline.

Source Configuration

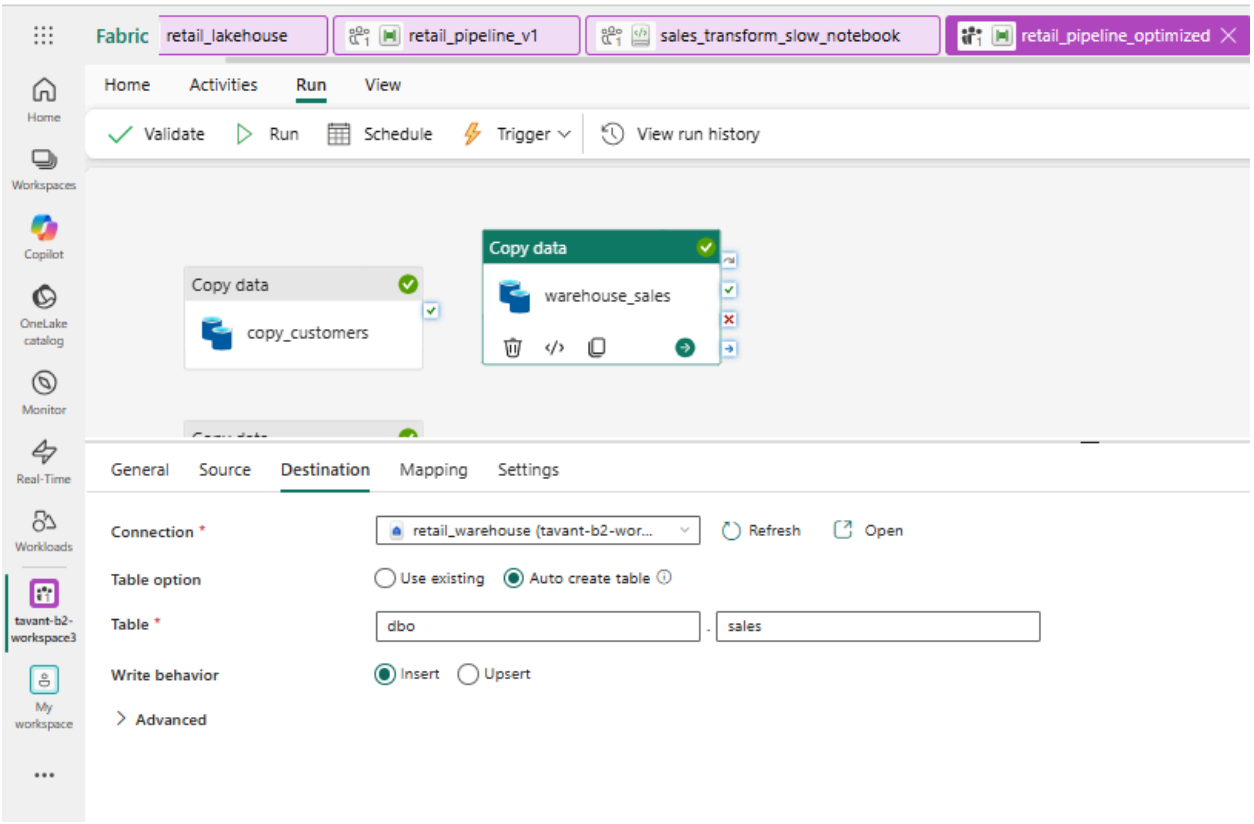
Field	Value
Source Type	Lakehouse Table
Connection	retail_lakehouse
Table	sales



Destination Configuration

Field	Value
Destination Type	Warehouse
Warehouse	retail_warehouse
Schema	dbo

Table	sales



Expected Output

Pipeline copies Lakehouse data into Warehouse table.

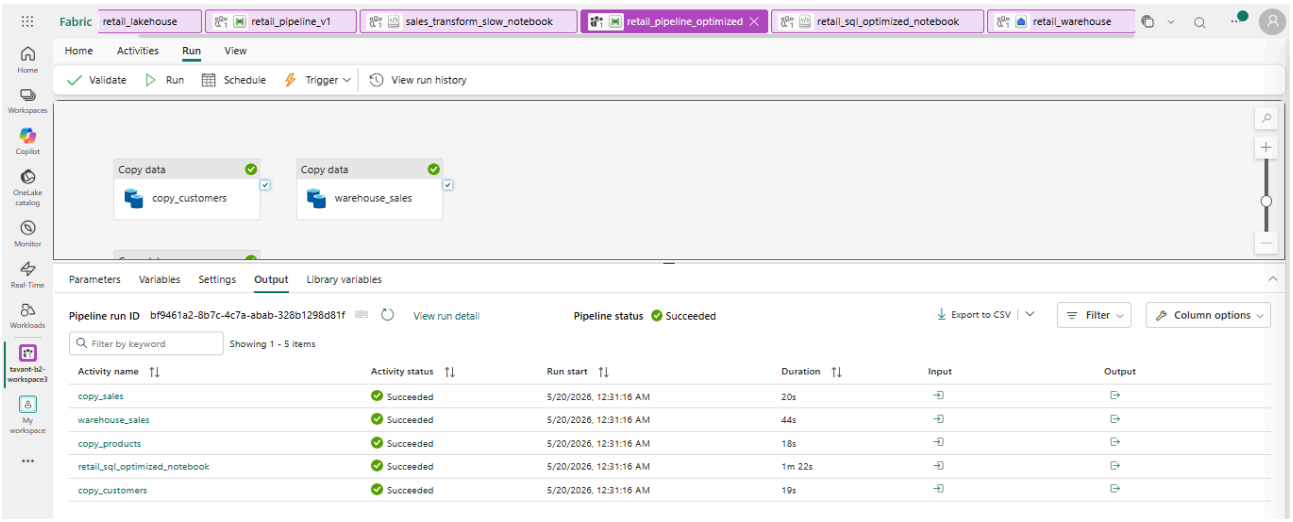
Run Pipeline

Click:

Run

Expected Output

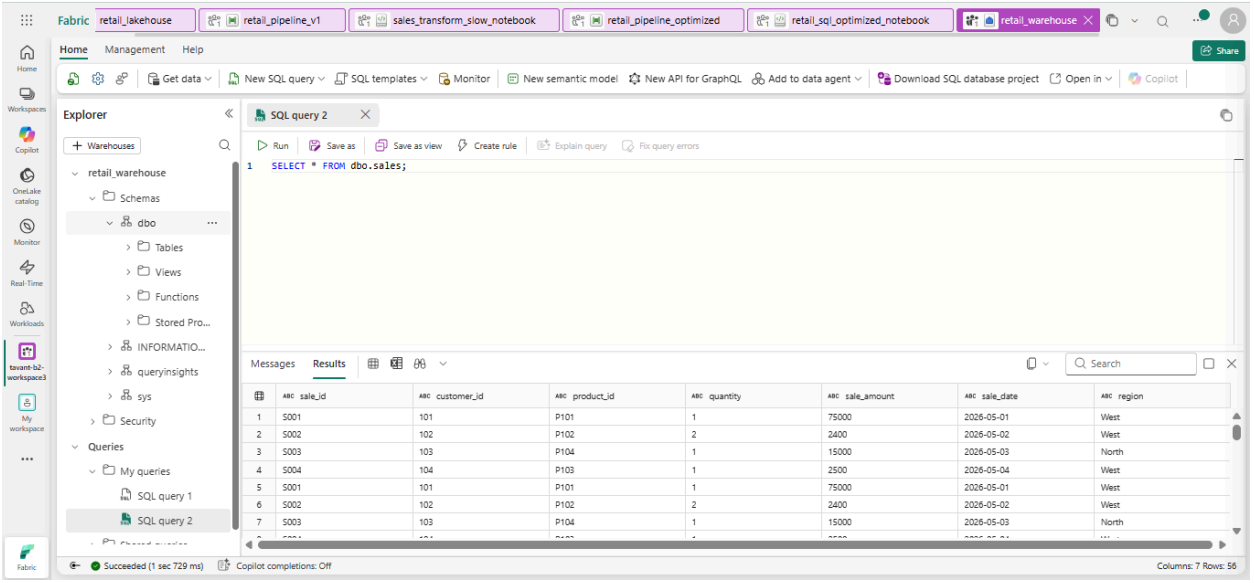
Copy activity succeeds.



Validate Warehouse Data

Inside Warehouse run:

SELECT * FROM dbo.sales;



Expected Output

Sales rows visible successfully.

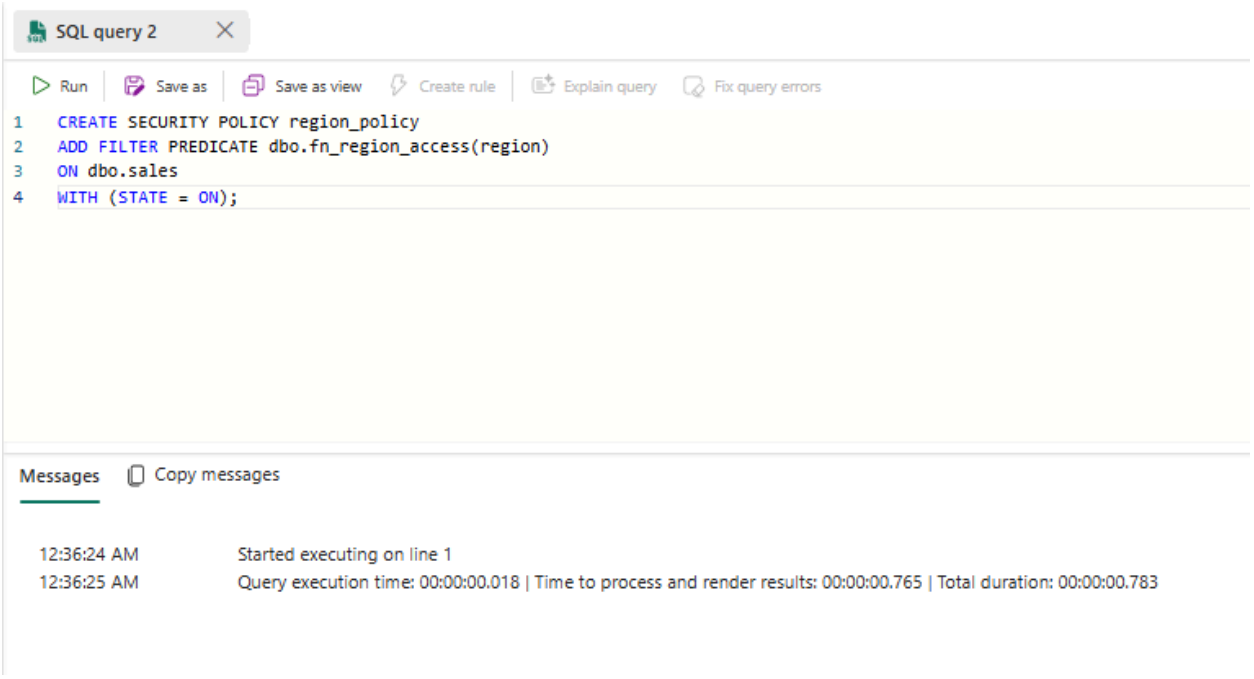
Step 25 — Apply Security Policy

CREATE SECURITY POLICY region_policy

ADD FILTER PREDICATE dbo.fn_region_access(region)

ON dbo.sales

WITH (STATE = ON);



Expected Output

Security policy active.

Step 26 — Validate Lineage

Open Lineage View

Actions

Workspace → Lineage

Expected Output

Visual dependency graph:

CSV Files

↓

Lakehouse

↓

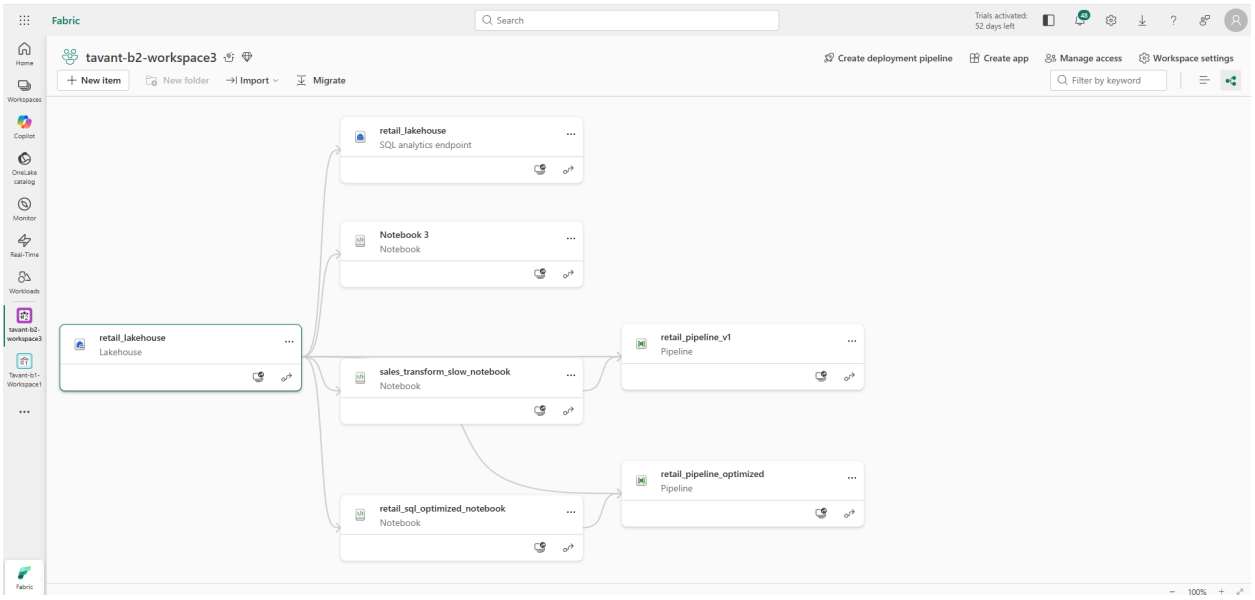
Pipeline

↓

Warehouse

↓

Reports



Step 27 — Monitor Metrics

Actions

Open:

Monitor Hub

Track:

- duration
- failures
- retries
- throughput

FINAL VALIDATION

Expected Final Assets

Asset	Expected
Lakehouse	Created
Tables	customers/products/sales
Pipelines	Slow + Optimized
Governance	Applied
Warehouse	Created
RLS	Working
Lineage	Visible
Monitoring	Active